

Notification Service Code Reference

Complete, tested implementation of every class, with the reasoning behind each design decision. Strategy, Observer and Decorator patterns on Spring Boot.

How to use this reference	Start here
1. Enums	Foundation
2. Domain model	Foundation
3. Exceptions	Foundation
4. DTOs	Web boundary
5. Decorator pattern	Decorator
6. Retry policy	Retry
7. Strategy pattern: channels and resolver	Strategy
8. Observer pattern: events and listeners	Observer
9. User preferences	Preferences
10. Notification engine	Orchestration
11. Notification service	Orchestration
12. Persistence	Persistence
13. Web layer and configuration	Web boundary
14. Tests	Verification
15. Common mistakes	Checklist

How to use this reference

This is the answer key, not the textbook. Write each class yourself first, using the UML page (docs/notification-service-uml.html). Open this only when you are stuck, read the matching class, close it, and write your own version. Your code can look different and still be correct.

Verified

Every line below was compiled and tested before this PDF was generated: **46 tests, 0 failures** (Spring Boot 4.1.0, Java 17 target, H2 in-memory database for tests). The best way to check *your* code is to copy a test class from the last chapter into your project and run it.

Package structure

```
org.example.notification.service
├── enums/           NotificationType, NotificationPriority, NotificationStatus
├── domain/          Notification
├── exception/       5 exceptions + GlobalExceptionHandler
├── dto/             request/NotificationRequest, response/NotificationResponse
├── decorator/       NotificationContent, SimpleNotification, NotificationDecorator,
                    TimestampNotificationDecorator, SignatureNotificationDecorator
├── retry/           RetryPolicy, ExponentialBackoffRetryPolicy
├── strategy/        NotificationChannel + Email / Sms / Push / InApp channels
├── factory/         NotificationChannelResolver
├── observer/        NotificationEvent, NotificationEventType, NotificationEventListener,
                    NotificationEventPublisher, Audit / Logging / Metrics listeners
├── preference/      NotificationPreferenceService, InMemoryNotificationPreferenceService
├── engine/          NotificationEngine
├── service/         NotificationService
├── entity/          NotificationEntity
├── repository/      NotificationRepository
│   └── jpa/         SpringDataNotificationRepository, JpaNotificationRepository
├── mapper/          NotificationMapper
├── controller/      NotificationController
└── config/          NotificationConfig
```

Chapter order = suggested build order

Each chapter only depends on earlier chapters, so you can compile and test as you go.

Project changes outside Java classes

Test dependencies for the repository and integration tests:

```
<!-- add inside <dependencies>, next to the other test dependencies -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa-test</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>test</scope>
</dependency>
```

Configuration used by NotificationConfig and NotificationService:

```
# append to src/main/resources/application.yml
```

```
notification:
```

```
  retry:
```

```
    max-attempts: 3
```

```
    initial-delay: 500ms
```

```
    multiplier: 2.0
```

```
    max-delay: 5s
```

```
  content:
```

```
    time-zone: UTC
```

```
    signature: "- Enterprise Digital Bank. Never share your OTP or password with anyone."
```

1. Enums

Start here. Enums have no dependencies, and every other class uses them. Notice that two of them carry a small piece of behaviour: rules that belong to the value itself.

NotificationType.java

src/main/java/.../enums/NotificationType.java · 8 lines

The four delivery mechanisms.

- Each value needs exactly one `NotificationChannel` bean. A type without a channel is not a crash: the engine marks the notification `FAILED`.
- Adding `WHATSAPP` later = add the value here + one new channel class. Nothing else changes.

```
package org.example.notificationsservice.enums;

public enum NotificationType {
    EMAIL,
    SMS,
    PUSH,
    IN_APP
}
```

NotificationPriority.java

src/main/java/.../enums/NotificationPriority.java · 16 lines

How urgent a notification is, plus one banking rule.

- `overridesUserPreferences()` puts the rule "CRITICAL ignores opt-out" on the enum, instead of scattering `if (priority == CRITICAL)` around the code.
- The service reads it as plain English: `notification.getPriority().overridesUserPreferences()`.

```
package org.example.notificationsservice.enums;

public enum NotificationPriority {
    LOW,
    MEDIUM,
    HIGH,
    CRITICAL;

    /**
     * Critical banking alerts (OTP, suspicious transaction, card blocked) must reach the
     * customer even if they switched that channel off.
     */
    public boolean overridesUserPreferences() {
        return this == CRITICAL;
    }
}
```

NotificationStatus.java

src/main/java/.../enums/NotificationStatus.java · 21 lines

The notification lifecycle as a state machine.

- `canTransitionTo(next)` lists every legal move. Anything else is rejected, so a bug can never move `SENT` back to `PROCESSING`.
- A `switch` is fine here: it describes data rules, it does not pick an implementation (that is what Strategy replaces).
- No `default` branch: if you add a new status, the compiler forces you to decide its transitions.

```
package org.example.notificationsservice.enums;

public enum NotificationStatus {
    CREATED,
    QUEUED,
    PROCESSING,
    SENT,
```

```
    FAILED,  
    RETRYING,  
    CANCELLED;  
  
    public boolean canTransitionTo(NotificationStatus next) {  
        return switch (this) {  
            case CREATED -> next == QUEUED || next == CANCELLED;  
            case QUEUED -> next == PROCESSING || next == FAILED || next == CANCELLED;  
            case PROCESSING -> next == SENT || next == RETRYING || next == FAILED;  
            case RETRYING -> next == PROCESSING || next == FAILED || next == CANCELLED;  
            case SENT, FAILED, CANCELLED -> false;  
        };  
    }  
}
```

2. Domain model

The notification as the application understands it. Plain Java: no JPA, no Spring. It protects its own rules, so no other class can put it into an invalid state.

Notification.java

src/main/java/.../domain/Notification.java · 77 lines

The central object that the service creates and the engine moves through its lifecycle.

- Fields that never change are `final`. Only `status` and `updatedAt` can change, and only through `transitionTo()`.
- The constructor is private. There are exactly two ways in: `create(...)` for a brand-new notification (generates a UUID, starts at `CREATED`) and `Notification.restore()...build()` for loading one from the database.
- `@Builder(builderMethodName = "restore")` is Lombok generating that builder. It avoids a 10-argument constructor where swapping `userId` and `title` would compile fine and be a silent bug.
- `now` is passed in rather than calling `Instant.now()` inside. That single choice makes every time-related test deterministic.

```
package org.example.notification.service.domain;

import lombok.Builder;
import lombok.Getter;
import org.example.notification.service.enums.NotificationPriority;
import org.example.notification.service.enums.NotificationStatus;
import org.example.notification.service.enums.NotificationType;

import java.time.Instant;
import java.util.Objects;
import java.util.UUID;

/**
 * A notification generated by the banking system.
 * Plain Java: no JPA annotations. The database shape lives in NotificationEntity.
 */
@Getter
public class Notification {

    private final String notificationId;
    private final String userId;
    private final String title;
    private final String content;
    private final NotificationType notificationType;
    private final NotificationPriority priority;
    private final String idempotencyKey;
    private final Instant createdAt;
    private NotificationStatus status;
    private Instant updatedAt;

    /**
     * Private: outside code uses create(...) for new notifications and
     * Notification.restore()...build() when loading one from the database.
     */
    @Builder(builderMethodName = "restore")
    private Notification(String notificationId,
                        String userId,
                        String title,
                        String content,
                        NotificationType notificationType,
                        NotificationPriority priority,
                        String idempotencyKey,
                        NotificationStatus status,
                        Instant createdAt,
                        Instant updatedAt) {
        this.notificationId = Objects.requireNonNull(notificationId, "notificationId");
        this.userId = Objects.requireNonNull(userId, "userId");
        this.title = Objects.requireNonNull(title, "title");
        this.content = Objects.requireNonNull(content, "content");
        this.notificationType = Objects.requireNonNull(notificationType, "notificationType");
        this.priority = Objects.requireNonNull(priority, "priority");
    }
}
```

```

        this.idempotencyKey = idempotencyKey;
        this.status = Objects.requireNonNull(status, "status");
        this.createdAt = Objects.requireNonNull(createdAt, "createdAt");
        this.updatedAt = Objects.requireNonNull(updatedAt, "updatedAt");
    }

    public static Notification create(String userId,
                                     String title,
                                     String content,
                                     NotificationType notificationType,
                                     NotificationPriority priority,
                                     String idempotencyKey,
                                     Instant now) {
        return new Notification(UUID.randomUUID().toString(), userId, title, content,
                                notificationType, priority, idempotencyKey, NotificationStatus.CREATED, now, now);
    }

    public void transitionTo(NotificationStatus next, Instant now) {
        if (!status.canTransitionTo(next)) {
            throw new IllegalStateException(
                "Notification %s cannot move from %s to %s".formatted(notificationId, status, next));
        }
        this.status = next;
        this.updatedAt = now;
    }
}

```

3. Exceptions

All custom exceptions are unchecked (`RuntimeException`), which is the normal style in Spring applications. Only two of them ever reach the HTTP client; the rest are handled inside the service and engine.

InvalidNotificationRequestException.java src/main/java/.../exception/InvalidNotificationRequestException.java · 17 lines

Request failed validation. Becomes HTTP 400.

- Keeps the full list of violations so the client sees every problem at once, not one at a time.
- `List.copyOf` makes the stored list immutable.

```
package org.example.notification.service.exception;

import java.util.List;

public class InvalidNotificationRequestException extends RuntimeException {

    private final List<String> violations;

    public InvalidNotificationRequestException(List<String> violations) {
        super("Invalid notification request: " + String.join("; ", violations));
        this.violations = List.copyOf(violations);
    }

    public List<String> getViolations() {
        return violations;
    }
}
```

NotificationNotFoundException.java src/main/java/.../exception/NotificationNotFoundException.java · 8 lines

Unknown notification id. Becomes HTTP 404.

```
package org.example.notification.service.exception;

public class NotificationNotFoundException extends RuntimeException {

    public NotificationNotFoundException(String notificationId) {
        super("Notification not found: " + notificationId);
    }
}
```

NotificationDeliveryException.java src/main/java/.../exception/NotificationDeliveryException.java · 27 lines

Thrown by channels when sending fails.

- Only the channel knows whether a failure is temporary (gateway timeout) or permanent (invalid phone number), so the channel decides via the `retryable` flag.
- Static factories `retryable(...)` and `permanent(...)` read far better at the call site than `new NotificationDeliveryException(msg, true, cause)`.

```
package org.example.notification.service.exception;

/**
 * Thrown by a channel when delivery fails. The channel knows whether the failure is
 * temporary (provider timeout, rate limit) or permanent (invalid phone number), so it decides.
 */
public class NotificationDeliveryException extends RuntimeException {

    private final boolean retryable;

    private NotificationDeliveryException(String message, boolean retryable, Throwable cause) {
        super(message, cause);
    }
}
```

```

        this.retryable = retryable;
    }

    public static NotificationDeliveryException retryable(String message, Throwable cause) {
        return new NotificationDeliveryException(message, true, cause);
    }

    public static NotificationDeliveryException permanent(String message, Throwable cause) {
        return new NotificationDeliveryException(message, false, cause);
    }

    public boolean isRetryable() {
        return retryable;
    }
}

```

UnsupportedNotificationChannelException.java

src/main/java/.../exception/UnsupportedNotificationChannelException.java
· 10 lines

No channel registered for a type. The engine turns it into **FAILED**.

```

package org.example.notification.service.exception;

import org.example.notification.service.enums.NotificationType;

public class UnsupportedNotificationChannelException extends RuntimeException {

    public UnsupportedNotificationChannelException(NotificationType notificationType) {
        super("No notification channel registered for type " + notificationType);
    }
}

```

DuplicateIdempotencyKeyException.java

src/main/java/.../exception/DuplicateIdempotencyKeyException.java · 8 lines

The database rejected a second row with the same idempotency key.

- The repository translates the low-level `DataIntegrityViolationException` into this, so the service never depends on Spring Data exception types.

```

package org.example.notification.service.exception;

public class DuplicateIdempotencyKeyException extends RuntimeException {

    public DuplicateIdempotencyKeyException(String idempotencyKey, Throwable cause) {
        super("A notification already exists for idempotency key " + idempotencyKey, cause);
    }
}

```

GlobalExceptionHandler.java

src/main/java/.../exception/GlobalExceptionHandler.java · 25 lines

Maps exceptions to HTTP responses for every controller.

- `ProblemDetail` is Spring's implementation of RFC 9457: a standard JSON error shape (`type`, `title`, `status`, `detail`) plus custom properties like `errors`.
- Only two handlers, because only two exceptions can escape the service.

```

package org.example.notification.service.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ProblemDetail;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice
public class GlobalExceptionHandler {

```

```
@ExceptionHandler(InvalidNotificationRequestException.class)
public ProblemDetail handleInvalidRequest(InvalidNotificationRequestException ex) {
    ProblemDetail problem = ProblemDetail.forStatusAndDetail(HttpStatus.BAD_REQUEST, "Request validation failed");
    problem.setTitle("Invalid notification request");
    problem.setProperty("errors", ex.getViolations());
    return problem;
}

@ExceptionHandler(NotificationNotFoundException.class)
public ProblemDetail handleNotFound(NotificationNotFoundException ex) {
    ProblemDetail problem = ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
    problem.setTitle("Notification not found");
    return problem;
}
}
```

4. DTOs

What crosses the API boundary. The domain `Notification` is never serialised to JSON directly.

NotificationRequest.java

src/main/java/.../dto/request/NotificationRequest.java · 20 lines

Input to `NotificationService.send()`.

- A `record`: immutable, with constructor, getters, `equals` and `toString` generated.
- Validation constraints live here. The service runs them, so callers that don't come through HTTP get the same checks.
- A good `idempotencyKey` comes from the banking event itself, e.g. `txn-9001:debit`. The same bank transaction then can never notify twice.

```
package org.example.notification.service.dto.request;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
import org.example.notification.service.enums.NotificationPriority;
import org.example.notification.service.enums.NotificationType;

/**
 * What a caller (controller today, banking event handlers later) sends to NotificationService.
 * idempotencyKey is optional; when present, the same key never creates a second notification.
 */
public record NotificationRequest(
    @NotBlank @Size(max = 64) String userId,
    @NotBlank @Size(max = 200) String title,
    @NotBlank @Size(max = 2000) String content,
    @NotNull NotificationType notificationType,
    @NotNull NotificationPriority priority,
    @Size(max = 128) String idempotencyKey) {
}
```

NotificationResponse.java

src/main/java/.../dto/response/NotificationResponse.java · 35 lines

Output returned by the controller.

- The `from(Notification)` static factory keeps the mapping in one place and the controller tiny.

```
package org.example.notification.service.dto.response;

import org.example.notification.service.domain.Notification;
import org.example.notification.service.enums.NotificationPriority;
import org.example.notification.service.enums.NotificationStatus;
import org.example.notification.service.enums.NotificationType;

import java.time.Instant;

public record NotificationResponse(
    String notificationId,
    String userId,
    String title,
    String content,
    NotificationType notificationType,
    NotificationPriority priority,
    NotificationStatus status,
    String idempotencyKey,
    Instant createdAt,
    Instant updatedAt) {

    public static NotificationResponse from(Notification notification) {
        return new NotificationResponse(
            notification.getNotificationId(),
            notification.getUserId(),
            notification.getTitle(),
            notification.getContent(),
            notification.getNotificationType(),
            notification.getNotificationPriority(),
            notification.getNotificationStatus(),
            notification.getIdempotencyKey(),
            notification.getCreatedAt(),
            notification.getUpdatedAt());
    }
}
```

```
        notification.getNotificationType(),  
        notification.getPriority(),  
        notification.getStatus(),  
        notification.getIdempotencyKey(),  
        notification.getCreatedAt(),  
        notification.getUpdatedAt());  
    }  
}
```

5. Decorator pattern

"I have an object. How do I add behaviour without changing its class?" Each decorator wraps another `NotificationContent` and adds one thing to the body. They stack in any order and any combination, with no subclass per combination.

NotificationContent.java

src/main/java/.../decorator/NotificationContent.java · 12 lines

The shared interface: the thing that gets decorated.

- Deliberately tiny. Everything that implements it (the simple notification and every decorator) is interchangeable.

```
package org.example.notification.service.decorator;

/**
 * The thing that gets decorated: the text of a notification.
 * SimpleNotification and every decorator implement this, so they are interchangeable.
 */
public interface NotificationContent {

    String getTitle();

    String getBody();

}
```

SimpleNotification.java

src/main/java/.../decorator/SimpleNotification.java · 25 lines

The innermost layer: original title and body.

- `Objects.requireNonNull` fails fast with a clear message instead of a mysterious `NullPointerException` later.

```
package org.example.notification.service.decorator;

import java.util.Objects;

/** The innermost layer: the original title and body, unchanged. */
public class SimpleNotification implements NotificationContent {

    private final String title;
    private final String body;

    public SimpleNotification(String title, String body) {
        this.title = Objects.requireNonNull(title, "title");
        this.body = Objects.requireNonNull(body, "body");
    }

    @Override
    public String getTitle() {
        return title;
    }

    @Override
    public String getBody() {
        return body;
    }

}
```

NotificationDecorator.java

src/main/java/.../decorator/NotificationDecorator.java · 26 lines

Abstract base class for all decorators.

- It **is** a `NotificationContent` (implements) and **has** one (the `wrapped` field). Those two facts together are the whole pattern.
- By default it forwards both methods to `wrapped`, so a subclass overrides only what it changes (here, always `getBody()`).

- `protected final wrapped`: subclasses can read it, nobody can replace it.

```
package org.example.notification.service.decorator;

import java.util.Objects;

/**
 * Base for all decorators. It IS a NotificationContent and HAS a NotificationContent.
 * By default it simply forwards to the wrapped object; subclasses override only what they change.
 */
public abstract class NotificationDecorator implements NotificationContent {

    protected final NotificationContent wrapped;

    protected NotificationDecorator(NotificationContent wrapped) {
        this.wrapped = Objects.requireNonNull(wrapped, "wrapped");
    }

    @Override
    public String getTitle() {
        return wrapped.getTitle();
    }

    @Override
    public String getBody() {
        return wrapped.getBody();
    }
}
```

TimestampNotificationDecorator.java

src/main/java/.../decorator/TimestampNotificationDecorator.java · 25 lines

Appends a formatted time line.

- It receives a `ZonedDateTime`, not a `Clock`: the decorator formats time but doesn't decide what time it is.
- The text is formatted once in the constructor, so calling `getBody()` twice gives the same result.
- `Locale.ENGLISH` keeps month names stable ("Sep") regardless of the server's language settings.

```
package org.example.notification.service.decorator;

import java.time.ZonedDateTime;
import java.time.format.DateTimeFormatter;
import java.util.Locale;

/** Appends the time the notification was generated, e.g. "Time: 16 Sep 2026, 10:15:32 UTC". */
public class TimestampNotificationDecorator extends NotificationDecorator {

    private static final DateTimeFormatter FORMAT =
        DateTimeFormatter.ofPattern("dd MMM yyyy, HH:mm:ss z", Locale.ENGLISH);

    private final String timestamp;

    public TimestampNotificationDecorator(NotificationContent wrapped, ZonedDateTime generatedAt) {
        super(wrapped);
        // Format once in the constructor so getBody() returns the same text every time it is called.
        this.timestamp = FORMAT.format(generatedAt);
    }

    @Override
    public String getBody() {
        return wrapped.getBody() + "\n\nTime: " + timestamp;
    }
}
```

Appends the bank's signature line.

- Trivial on purpose. A future `MaskAccountNumberDecorator` or `DisclaimerDecorator` has exactly the same shape.

```
package org.example.notification.service.decorator;

import java.util.Objects;

/** Appends the bank's signature line to the body. */
public class SignatureNotificationDecorator extends NotificationDecorator {

    private final String signature;

    public SignatureNotificationDecorator(NotificationContent wrapped, String signature) {
        super(wrapped);
        this.signature = Objects.requireNonNull(signature, "signature");
    }

    @Override
    public String getBody() {
        return wrapped.getBody() + "\n\n" + signature;
    }
}
```

6. Retry policy

A small abstraction the engine asks two questions: *should I retry this failure?* and *how long should I wait?* Swapping in a fixed-delay policy later means writing one class and changing one bean.

RetryPolicy.java

src/main/java/.../retry/RetryPolicy.java · 15 lines

The contract the engine depends on.

- `maxAttempts` counts the first attempt too: 3 means try, retry, retry.

```
package org.example.notificationsservice.retry;

import java.time.Duration;

public interface RetryPolicy {

    /** Total attempts allowed, including the first one. 3 means: try, retry, retry. */
    int maxAttempts();

    /** Is it worth trying again after this failure? */
    boolean isRetryable(Throwable failure);

    /** How long to wait after the given failed attempt (1-based) before the next one. */
    Duration delayAfterAttempt(int failedAttempt);
}
```

ExponentialBackoffRetryPolicy.java

src/main/java/.../retry/ExponentialBackoffRetryPolicy.java · 61 lines

Waits longer after each failure, up to a cap.

- The constructor validates its configuration, so a typo in `application.yml` fails at startup instead of during a real outage.
- `failure instanceof NotificationDeliveryException delivery && delivery.isRetryable()` is Java 16+ pattern matching: test the type and bind a variable in one step.
- Unexpected exceptions (`NullPointerException`, etc.) are not retried: they are bugs, and retrying a bug just fails three times.
- `Math.pow` can overflow to `Infinity` for huge attempt numbers; comparing against `maxDelay` first handles that safely.
- Not annotated as a Spring bean. It is created in `NotificationConfig` so its numbers come from configuration.

```
package org.example.notificationsservice.retry;

import org.example.notificationsservice.exception.NotificationDeliveryException;

import java.time.Duration;
import java.util.Objects;

/**
 * Waits longer after each failure: initialDelay, initialDelay * multiplier, ... capped at maxDelay.
 * Example (500ms, x2, max 5s): 500ms, 1s, 2s, 4s, 5s, 5s ...
 */
public class ExponentialBackoffRetryPolicy implements RetryPolicy {

    private final int maxAttempts;
    private final Duration initialDelay;
    private final double multiplier;
    private final Duration maxDelay;

    public ExponentialBackoffRetryPolicy(int maxAttempts, Duration initialDelay, double multiplier, Duration maxDelay) {
        if (maxAttempts < 1) {
            throw new IllegalArgumentException("maxAttempts must be at least 1");
        }
        if (Objects.requireNonNull(initialDelay, "initialDelay").isNegative()) {
            throw new IllegalArgumentException("initialDelay must not be negative");
        }
        if (multiplier < 1.0) {
            throw new IllegalArgumentException("multiplier must be at least 1.0");
        }
    }
}
```

```

    }
    if (Objects.requireNonNull(maxDelay, "maxDelay").compareTo(initialDelay) < 0) {
        throw new IllegalArgumentException("maxDelay must be >= initialDelay");
    }
    this.maxAttempts = maxAttempts;
    this.initialDelay = initialDelay;
    this.multiplier = multiplier;
    this.maxDelay = maxDelay;
}

@Override
public int maxAttempts() {
    return maxAttempts;
}

@Override
public boolean isRetryable(Throwable failure) {
    // Only failures a channel explicitly marked as temporary are retried.
    // Anything else (NullPointerException, IllegalStateException...) is a bug: retrying won't fix it.
    return failure instanceof NotificationDeliveryException delivery && delivery.isRetryable();
}

@Override
public Duration delayAfterAttempt(int failedAttempt) {
    if (failedAttempt < 1) {
        throw new IllegalArgumentException("failedAttempt must be at least 1");
    }
    double millis = initialDelay.toMillis() * Math.pow(multiplier, failedAttempt - 1);
    if (millis >= maxDelay.toMillis()) { // also covers Infinity for very large attempts
        return maxDelay;
    }
    return Duration.ofMillis((long) millis);
}
}

```

7. Strategy pattern: channels and resolver

"I need to send this. Which implementation should do it?" One class per delivery mechanism, all behind one interface. The resolver picks the right one with a map lookup: no `if/else`, no `switch`.

NotificationChannel.java

src/main/java/.../strategy/NotificationChannel.java · 20 lines

The strategy interface.

- `supportedType()` lets each channel declare its own key, so the resolver never needs to know any concrete class name.

```
package org.example.notificationsservice.strategy;

import org.example.notificationsservice.domain.Notification;
import org.example.notificationsservice.enums.NotificationType;

/**
 * Strategy interface: one implementation per delivery mechanism.
 */
public interface NotificationChannel {

    /** Which NotificationType this channel delivers. The resolver uses it as the map key. */
    NotificationType supportedType();

    /**
     * Deliver the notification.
     *
     * @throws org.example.notificationsservice.exception.NotificationDeliveryException when delivery fails
     */
    void send(Notification notification);
}
```

EmailNotificationChannel.java

src/main/java/.../strategy/EmailNotificationChannel.java · 25 lines

Email delivery (simulated).

- Delivery is a log line for now, because this service doesn't yet know the user's email address (see *Open decisions* on the UML page).
- The comment shows where `JavaMailSender` goes and how to wrap provider errors.
- Logs the id and user, **never** the content: bank notifications contain OTPs and balances.

```
package org.example.notificationsservice.strategy;

import lombok.extern.slf4j.Slf4j;
import org.example.notificationsservice.domain.Notification;
import org.example.notificationsservice.enums.NotificationType;
import org.springframework.stereotype.Component;

@Slf4j
@Component
public class EmailNotificationChannel implements NotificationChannel {

    @Override
    public NotificationType supportedType() {
        return NotificationType.EMAIL;
    }

    @Override
    public void send(Notification notification) {
        // Simulated delivery. Real version: Look up the user's email address, then call JavaMailSender.
        // Wrap provider errors in NotificationDeliveryException.retryable(...) or .permanent(...).
        // Never Log notification.getContent(): it can contain an OTP.
        log.info("[EMAIL] Delivered notification {} to user {}",
            notification.getNotificationId(), notification.getUserId());
    }
}
```

SMS delivery (simulated).

```
package org.example.notificationsservice.strategy;

import lombok.extern.slf4j.Slf4j;
import org.example.notificationsservice.domain.Notification;
import org.example.notificationsservice.enums.NotificationType;
import org.springframework.stereotype.Component;

@Slf4j
@Component
public class SmsNotificationChannel implements NotificationChannel {

    @Override
    public NotificationType supportedType() {
        return NotificationType.SMS;
    }

    @Override
    public void send(Notification notification) {
        // Simulated delivery. Real version: call an SMS gateway (Twilio, MSG91, ...) with the user's phone number.
        log.info("[SMS] Delivered notification {} to user {}",
            notification.getNotificationId(), notification.getUserId());
    }
}
```

PushNotificationChannel.java

src/main/java/.../strategy/PushNotificationChannel.java · 23 lines

Push delivery (simulated).

```
package org.example.notificationsservice.strategy;

import lombok.extern.slf4j.Slf4j;
import org.example.notificationsservice.domain.Notification;
import org.example.notificationsservice.enums.NotificationType;
import org.springframework.stereotype.Component;

@Slf4j
@Component
public class PushNotificationChannel implements NotificationChannel {

    @Override
    public NotificationType supportedType() {
        return NotificationType.PUSH;
    }

    @Override
    public void send(Notification notification) {
        // Simulated delivery. Real version: send to Firebase Cloud Messaging / APNs using the user's device tokens.
        log.info("[PUSH] Delivered notification {} to user {}",
            notification.getNotificationId(), notification.getUserId());
    }
}
```

InAppNotificationChannel.java

src/main/java/.../strategy/InAppNotificationChannel.java · 24 lines

In-app inbox.

- Interesting case: the notification is already saved, so saving is delivery. The app reads it with `GET /api/v1/notifications?userId=...`.

```
package org.example.notificationsservice.strategy;

import lombok.extern.slf4j.Slf4j;
import org.example.notificationsservice.domain.Notification;
import org.example.notificationsservice.enums.NotificationType;
import org.springframework.stereotype.Component;
```

```

@Slf4j
@Component
public class InAppNotificationChannel implements NotificationChannel {

    @Override
    public NotificationType supportedType() {
        return NotificationType.IN_APP;
    }

    @Override
    public void send(Notification notification) {
        // The notification is already saved, so the banking app can fetch it with
        // GET /api/v1/notifications?userId=... . A real version might also push it over a WebSocket.
        log.info("[IN_APP] Notification {} is available in the inbox of user {}",
            notification.getNotificationId(), notification.getUserId());
    }
}

```

NotificationChannelResolver.java

src/main/java/.../factory/NotificationChannelResolver.java · 44 lines

Maps a `NotificationType` to its channel.

- Spring injects **all** `NotificationChannel` beans as a `List`. Add a new `@Component` channel and it's picked up automatically: that's the Open/Closed principle in practice.
- `EnumMap` is the fastest, most compact map when keys are enum values.
- Two channels claiming the same type throws at startup. Failing loudly beats silently choosing one.
- `resolve()` throws our own exception, which the engine converts into a `FAILED` status.

```

package org.example.notification.service.factory;

import org.example.notification.service.enums.NotificationType;
import org.example.notification.service.exception.UnsupportedNotificationChannelException;
import org.example.notification.service.strategy.NotificationChannel;
import org.springframework.stereotype.Component;

import java.util.Collections;
import java.util.EnumMap;
import java.util.List;
import java.util.Map;

/**
 * Picks the right NotificationChannel for a NotificationType without any if/else or switch.
 * Spring passes in every NotificationChannel bean it finds.
 */
@Component
public class NotificationChannelResolver {

    private final Map<NotificationType, NotificationChannel> channelsByType;

    public NotificationChannelResolver(List<NotificationChannel> channels) {
        Map<NotificationType, NotificationChannel> map = new EnumMap<>(NotificationType.class);
        for (NotificationChannel channel : channels) {
            NotificationChannel previous = map.putIfAbsent(channel.supportedType(), channel);
            if (previous != null) {
                // Fail at startup rather than silently picking one of two channels.
                throw new IllegalStateException("Two channels registered for %s: %s and %s".formatted(
                    channel.supportedType(),
                    previous.getClass().getSimpleName(),
                    channel.getClass().getSimpleName()));
            }
        }
        this.channelsByType = Collections.unmodifiableMap(map);
    }

    public NotificationChannel resolve(NotificationType notificationType) {
        NotificationChannel channel = channelsByType.get(notificationType);
        if (channel == null) {
            throw new UnsupportedNotificationChannelException(notificationType);
        }
    }
}

```

```
    return channel;  
  }  
}
```

8. Observer pattern: events and listeners

"Something happened. Who needs to know?" The service and engine publish events without knowing who listens. Audit, logging and metrics react independently, and adding a fourth listener changes nothing else. (Spring has a built-in version, `ApplicationEventPublisher` + `@EventListener`. Writing it by hand here makes the pattern visible.)

NotificationEventType.java

src/main/java/.../observer/NotificationEventType.java · 9 lines

What can happen to a notification.

```
package org.example.notification.service.observer;

public enum NotificationEventType {
    NOTIFICATION_CREATED,
    NOTIFICATION_SENT,
    NOTIFICATION_RETRYING,
    NOTIFICATION_FAILED,
    NOTIFICATION_CANCELLED
}
```

NotificationEvent.java

src/main/java/.../observer/NotificationEvent.java · 42 lines

An immutable snapshot passed to listeners.

- A `record`, so listeners physically cannot change it. They never receive the real `Notification`, so they can't change its status either.
- It intentionally has no `title` or `content`: listeners write logs, and logs must not contain OTPs.
- Two `of(...)` overloads: with and without a `reason`.

```
package org.example.notification.service.observer;

import org.example.notification.service.domain.Notification;
import org.example.notification.service.enums.NotificationPriority;
import org.example.notification.service.enums.NotificationStatus;
import org.example.notification.service.enums.NotificationType;

import java.time.Instant;

/**
 * Immutable snapshot of a notification at the moment something happened.
 * Deliberately has no title/content: listeners log and audit, and the body may contain an OTP.
 */
public record NotificationEvent(
    NotificationEventType eventType,
    String notificationId,
    String userId,
    NotificationType notificationType,
    NotificationPriority priority,
    NotificationStatus status,
    String reason,
    Instant occurredAt) {

    public static NotificationEvent of(NotificationEventType eventType, Notification notification, Instant occurredAt) {
        return of(eventType, notification, null, occurredAt);
    }

    public static NotificationEvent of(NotificationEventType eventType,
                                       Notification notification,
                                       String reason,
                                       Instant occurredAt) {
        return new NotificationEvent(
            eventType,
            notification.getNotificationId(),
            notification.getUserId(),
            notification.getNotificationType(),
            notification.getPriority(),
            notification.getStatus(),
            reason,
            occurredAt);
    }
}
```

```

        notification.getStatus(),
        reason,
        occurredAt);
    }
}

```

NotificationEventListener.java

src/main/java/.../observer/NotificationEventListener.java · 7 lines

The observer interface.

- One abstract method makes it a functional interface, so tests can pass a lambda: `event -> calls.add(...)`.

```

package org.example.notification.service.observer;

/** Observer interface. Anything that wants to react to notification events implements this. */
public interface NotificationEventListener {

    void onEvent(NotificationEvent event);
}

```

NotificationEventPublisher.java

src/main/java/.../observer/NotificationEventPublisher.java · 43 lines

The subject: keeps listeners and notifies all of them.

- The constructor receives every listener bean from Spring. `subscribe / unsubscribe` exist for the classic Observer API and for tests.
- `CopyOnWriteArrayList` is ideal when reads (publishing) vastly outnumber writes (subscribing): iteration is always safe, even during a concurrent subscribe.
- The per-listener `try/catch` is the most important line: a broken metrics listener must never stop an SMS from being marked SENT.

```

package org.example.notification.service.observer;

import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Component;

import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;

/**
 * Subject / observable. Keeps the list of listeners and tells all of them about every event.
 */
@Slf4j
@Component
public class NotificationEventPublisher {

    // CopyOnWriteArrayList: safe to iterate while another thread subscribes or unsubscribes.
    private final CopyOnWriteArrayList<NotificationEventListener> listeners = new CopyOnWriteArrayList<>();

    /** Spring passes every NotificationEventListener bean, so the three listeners register automatically. */
    public NotificationEventPublisher(List<NotificationEventListener> listeners) {
        this.listeners.addAll(listeners);
    }

    public void subscribe(NotificationEventListener listener) {
        listeners.addIfAbsent(listener);
    }

    public void unsubscribe(NotificationEventListener listener) {
        listeners.remove(listener);
    }

    public void publish(NotificationEvent event) {
        for (NotificationEventListener listener : listeners) {
            try {
                listener.onEvent(event);
            } catch (RuntimeException ex) {
                // One broken listener must not stop the others or break delivery.
            }
        }
    }
}

```

```

        log.error("Listener {} failed on {} for notification {}",
            listener.getClass().getSimpleName(), event.eventType(), event.notificationId(), ex);
    }
}
}
}

```

NotificationAuditListener.java

src/main/java/.../observer/NotificationAuditListener.java · 23 lines

Compliance-style audit trail.

- Uses a dedicated logger name, `NOTIFICATION_AUDIT`, so logback can route it to its own file. Later it could call the banking system's audit module.

```

package org.example.notificationsservice.observer;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;

/**
 * Writes a stable, one-line audit record for every lifecycle event.
 * Uses its own logger name so Logback can route it to a separate audit file.
 * Later this could call the banking system's audit module instead.
 */
@Component
public class NotificationAuditListener implements NotificationEventListener {

    private static final Logger AUDIT = LoggerFactory.getLogger("NOTIFICATION_AUDIT");

    @Override
    public void onEvent(NotificationEvent event) {
        AUDIT.info("event={} notificationId={} userId={} channel={} priority={} status={} reason={} at={}",
            event.eventType(), event.notificationId(), event.userId(), event.notificationType(),
            event.priority(), event.status(), event.reason(), event.occurredAt());
    }
}

```

NotificationLoggingListener.java

src/main/java/.../observer/NotificationLoggingListener.java · 22 lines

Operational logs for developers.

- Failures and retries log at `WARN`, so they stand out.

```

package org.example.notificationsservice.observer;

import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Component;

/** Operational Logs for developers: failures as WARN, everything else as INFO. */
@Slf4j
@Component
public class NotificationLoggingListener implements NotificationEventListener {

    @Override
    public void onEvent(NotificationEvent event) {
        if (event.eventType() == NotificationEventType.NOTIFICATION_FAILED
            || event.eventType() == NotificationEventType.NOTIFICATION_RETRYING) {
            log.warn("{} for notification {} ({}): {}",
                event.eventType(), event.notificationId(), event.notificationType(), event.reason());
        } else {
            log.info("{} for notification {} ({})",
                event.eventType(), event.notificationId(), event.notificationType());
        }
    }
}

```

Counts events by type.

- The `EnumMap` is filled completely in the constructor and never modified afterwards, so reading it from many threads is safe.
- `LongAdder` is a counter built for many threads incrementing at once.
- When you add Actuator, replace the counters with Micrometer `Counter`s. The listener shape stays the same.

```
package org.example.notification.service.observer;

import org.springframework.stereotype.Component;

import java.util.EnumMap;
import java.util.Map;
import java.util.concurrent.atomic.LongAdder;

/**
 * Counts events by type. Kept dependency-free for now; swap the LongAdders for
 * Micrometer counters when you add Spring Boot Actuator.
 */
@Component
public class NotificationMetricsListener implements NotificationEventListener {

    private final Map<NotificationEventType, LongAdder> counters = new EnumMap<>(NotificationEventType.class);

    public NotificationMetricsListener() {
        // Fill every key up front so the map is never modified afterwards (thread-safe reads).
        for (NotificationEventType type : NotificationEventType.values()) {
            counters.put(type, new LongAdder());
        }
    }

    @Override
    public void onEvent(NotificationEvent event) {
        counters.get(event.eventType()).increment();
    }

    public long count(NotificationEventType type) {
        return counters.get(type).sum();
    }
}
```

9. User preferences

A deliberately small abstraction: the service only needs to ask one yes/no question.

NotificationPreferenceService.java

src/main/java/.../preference/NotificationPreferenceService.java · 8 lines

The question the service asks.

```
package org.example.notification.service.preference;

import org.example.notification.service.enums.NotificationType;

public interface NotificationPreferenceService {

    boolean isChannelEnabled(String userId, NotificationType notificationType);

}
```

InMemoryNotificationPreferenceService.java

src/main/java/.../preference/InMemoryNotificationPreferenceService.java · 39 lines

Simple opt-out store.

- Every channel is enabled unless the user turned it off.
- `ConcurrentHashMap.compute` updates one user atomically. A fresh `EnumSet` copy is stored each time, so readers never see a set mid-change.
- In memory only: preferences vanish on restart. A JPA-backed implementation can replace it without touching the service.

```
package org.example.notification.service.preference;

import org.example.notification.service.enums.NotificationType;
import org.springframework.stereotype.Service;

import java.util.EnumSet;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;

/**
 * Every channel is enabled unless the user switched it off (opt-out).
 * Stored in memory only: preferences are lost on restart. Replace with a JPA version later.
 */
@Service
public class InMemoryNotificationPreferenceService implements NotificationPreferenceService {

    private final Map<String, Set<NotificationType>> disabledChannelsByUser = new ConcurrentHashMap<>();

    @Override
    public boolean isChannelEnabled(String userId, NotificationType notificationType) {
        return !disabledChannelsByUser.getOrDefault(userId, Set.of()).contains(notificationType);
    }

    public void setChannelEnabled(String userId, NotificationType notificationType, boolean enabled) {
        // compute() is atomic per key; we always store a fresh copy so readers never see a set being changed.
        disabledChannelsByUser.compute(userId, (id, disabled) -> {
            Set<NotificationType> updated = disabled == null
                ? EnumSet.noneOf(NotificationType.class)
                : EnumSet.copyOf(disabled);
            if (enabled) {
                updated.remove(notificationType);
            } else {
                updated.add(notificationType);
            }
            return updated.isEmpty() ? null : updated; // returning null removes the entry
        });
    }
}
```

10. Notification engine

Delivers one QUEUED notification. It combines Strategy (resolver + channel), the retry policy, and Observer (publishing results), yet contains no knowledge of email, SMS or push.

NotificationEngine.java

src/main/java/.../engine/NotificationEngine.java · 119 lines

Resolve the channel, attempt delivery, retry, record every status.

- Trace one run: PROCESSING → send() → success → SENT. On failure: ask the policy → RETRYING → wait → PROCESSING again, or FAILED.
- trySend() returns the exception instead of throwing it. That keeps the loop flat and readable.
- Saving SENT happens **outside** the try/catch on purpose. If the database write failed inside the catch block, the loop would send the SMS a second time.
- Every RuntimeException from a channel is caught, so a notification can never get stuck in PROCESSING.
- Thread.sleep blocks the request thread (about 1.5s worst case with default settings). That's acceptable for v1. Later, move delivery to @Async or a queue.
- On InterruptedException, the interrupt flag is restored: the standard rule, so the caller still knows the thread was interrupted.

```
package org.example.notificationengine;

import org.example.notificationengine.domain.Notification;
import org.example.notificationengine.enums.NotificationStatus;
import org.example.notificationengine.exception.UnsupportedNotificationChannelException;
import org.example.notificationengine.factory.NotificationChannelResolver;
import org.example.notificationengine.observer.NotificationEvent;
import org.example.notificationengine.observer.NotificationEventPublisher;
import org.example.notificationengine.observer.NotificationEventType;
import org.example.notificationengine.repository.NotificationRepository;
import org.example.notificationengine.retry.RetryPolicy;
import org.example.notificationengine.strategy.NotificationChannel;
import org.springframework.stereotype.Component;

import java.time.Clock;
import java.time.Duration;

/**
 * Delivers a QUEUED notification: pick the channel, try to send, retry when the policy allows,
 * and record every status change. Knows nothing about Email/SMS/Push specifically.
 */
@Component
public class NotificationEngine {

    private final NotificationChannelResolver channelResolver;
    private final NotificationRepository repository;
    private final NotificationEventPublisher eventPublisher;
    private final RetryPolicy retryPolicy;
    private final Clock clock;

    public NotificationEngine(NotificationChannelResolver channelResolver,
                             NotificationRepository repository,
                             NotificationEventPublisher eventPublisher,
                             RetryPolicy retryPolicy,
                             Clock clock) {
        this.channelResolver = channelResolver;
        this.repository = repository;
        this.eventPublisher = eventPublisher;
        this.retryPolicy = retryPolicy;
        this.clock = clock;
    }

    public void deliver(Notification notification) {
        NotificationChannel channel;
        try {
            channel = channelResolver.resolve(notification.getNotificationType());
        } catch (UnsupportedNotificationChannelException ex) {
```

```

        markFailed(notification, ex);
        return;
    }

    for (int attempt = 1; ; attempt++) {
        changeStatus(notification, NotificationStatus.PROCESSING);

        RuntimeException failure = trySend(channel, notification);
        if (failure == null) {
            // Kept outside the try/catch on purpose: if saving SENT fails we must NOT send again.
            changeStatus(notification, NotificationStatus.SENT);
            publish(NotificationEventType.NOTIFICATION_SENT, notification, null);
            return;
        }

        boolean attemptsLeft = attempt < retryPolicy.maxAttempts();
        if (!attemptsLeft || !retryPolicy.isRetryable(failure)) {
            markFailed(notification, failure);
            return;
        }

        changeStatus(notification, NotificationStatus.RETRYING);
        publish(NotificationEventType.NOTIFICATION_RETRYING, notification, reasonOf(failure));

        if (!waitBeforeRetry(retryPolicy.delayAfterAttempt(attempt))) {
            markFailed(notification, failure);
            return;
        }
    }
}

/** Returns null on success, or the exception the channel threw. */
private RuntimeException trySend(NotificationChannel channel, Notification notification) {
    try {
        channel.send(notification);
        return null;
    } catch (RuntimeException ex) {
        return ex;
    }
}

private boolean waitBeforeRetry(Duration delay) {
    if (delay.isZero() || delay.isNegative()) {
        return true;
    }
    try {
        Thread.sleep(delay.toMillis());
        return true;
    } catch (InterruptedException ex) {
        Thread.currentThread().interrupt(); // keep the interrupt flag for whoever called us
        return false;
    }
}

private void markFailed(Notification notification, RuntimeException failure) {
    changeStatus(notification, NotificationStatus.FAILED);
    publish(NotificationEventType.NOTIFICATION_FAILED, notification, reasonOf(failure));
}

private void changeStatus(Notification notification, NotificationStatus status) {
    notification.transitionTo(status, clock.instant());
    repository.updateStatus(notification);
}

private void publish(NotificationEventType type, Notification notification, String reason) {
    eventPublisher.publish(NotificationEvent.of(type, notification, reason, clock.instant()));
}

private static String reasonOf(RuntimeException failure) {
    return failure.getMessage() != null ? failure.getMessage() : failure.getClass().getSimpleName();
}
}

```

11. Notification service

The application's entry point for notifications. It orchestrates the steps in order but performs no delivery itself.

NotificationService.java

src/main/java/.../service/NotificationService.java · 151 lines

Validate, deduplicate, decorate, persist, publish, check preferences, hand to the engine.

- Numbered comments inside `send()` match the flow on the UML page.
- A concrete class, not interface + `Impl`. There is only one implementation, so an interface would add a file without adding flexibility.
- Validation uses the injected `jakarta.validation.Validator` rather than `@Valid` on the controller, so future non-HTTP callers (Kafka consumers) get identical checks.
- Blank idempotency keys become `null`, so `" "` can't accidentally deduplicate unrelated requests.
- Race condition handled: two identical requests can both see "no existing key". The database's unique constraint rejects the second, and the service returns the winner.
- Decorators run **before** saving, so the database stores exactly the text the customer received.
- Not `@Transactional`: holding a database transaction open while an SMS gateway responds (or while retry sleeps) would exhaust the connection pool under load.
- Constructor takes two extra dependencies compared with the UML: `clock` (testable time) and the signature text from configuration.

```
package org.example.notificationsservice.service;

import jakarta.validation.ConstraintViolation;
import jakarta.validation.Validator;
import lombok.extern.slf4j.Slf4j;
import org.example.notificationsservice.decorator.NotificationContent;
import org.example.notificationsservice.decorator.SignatureNotificationDecorator;
import org.example.notificationsservice.decorator.SimpleNotification;
import org.example.notificationsservice.decorator.TimestampNotificationDecorator;
import org.example.notificationsservice.domain.Notification;
import org.example.notificationsservice.dto.request.NotificationRequest;
import org.example.notificationsservice.engine.NotificationEngine;
import org.example.notificationsservice.enums.NotificationStatus;
import org.example.notificationsservice.exception.DuplicateIdempotencyKeyException;
import org.example.notificationsservice.exception.InvalidNotificationRequestException;
import org.example.notificationsservice.exception.NotificationNotFoundException;
import org.example.notificationsservice.observer.NotificationEvent;
import org.example.notificationsservice.observer.NotificationEventPublisher;
import org.example.notificationsservice.observer.NotificationEventType;
import org.example.notificationsservice.preference.NotificationPreferenceService;
import org.example.notificationsservice.repository.NotificationRepository;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import org.springframework.util.StringUtils;

import java.time.Clock;
import java.time.ZonedDateTime;
import java.util.List;
import java.util.Optional;
import java.util.Set;

/**
 * Application entry point for notifications. Orchestrates; never delivers anything itself.
 * Not @Transactional on purpose: we don't want a DB transaction open while a channel is sending.
 */
@Slf4j
@Service
public class NotificationService {

    private final NotificationRepository repository;
    private final NotificationEngine engine;
    private final NotificationEventPublisher eventPublisher;
    private final NotificationPreferenceService preferenceService;
```

```

private final Validator validator;
private final Clock clock;
private final String signature;

public NotificationService(NotificationRepository repository,
                           NotificationEngine engine,
                           NotificationEventPublisher eventPublisher,
                           NotificationPreferenceService preferenceService,
                           Validator validator,
                           Clock clock,
                           @Value("${notification.content.signature:}") String signature) {
    this.repository = repository;
    this.engine = engine;
    this.eventPublisher = eventPublisher;
    this.preferenceService = preferenceService;
    this.validator = validator;
    this.clock = clock;
    this.signature = signature;
}

public Notification send(NotificationRequest request) {
    validate(request);
    String idempotencyKey = StringUtils.hasText(request.idempotencyKey()) ? request.idempotencyKey() : null;

    // 1. Same key seen before? Return the original, never send twice.
    if (idempotencyKey != null) {
        Optional<Notification> existing = repository.findByIdempotencyKey(idempotencyKey);
        if (existing.isPresent()) {
            log.info("Idempotency key already used, returning notification {}", existing.get().getNotificationId());
            return existing.get();
        }
    }

    // 2. Build the final text with decorators, then the domain object.
    NotificationContent content = composeContent(request);
    Notification notification = Notification.create(
        request.userId(),
        content.getTitle(),
        content.getBody(),
        request.notificationType(),
        request.priority(),
        idempotencyKey,
        clock.instant());

    // 3. Persist. A concurrent request with the same key may have inserted first.
    try {
        repository.save(notification);
    } catch (DuplicateIdempotencyKeyException ex) {
        return repository.findByIdempotencyKey(idempotencyKey).orElseThrow(() -> ex);
    }
    eventPublisher.publish(NotificationEvent.of(NotificationEventType.NOTIFICATION_CREATED, notification,
        clock.instant()));

    // 4. Respect user preferences, except for CRITICAL notifications.
    if (!isDeliveryAllowed(notification)) {
        changeStatus(notification, NotificationStatus.CANCELLED);
        eventPublisher.publish(NotificationEvent.of(NotificationEventType.NOTIFICATION_CANCELLED, notification,
            "Channel %s disabled by user".formatted(notification.getNotificationType()), clock.instant()));
        return notification;
    }

    // 5. Hand over to the engine.
    changeStatus(notification, NotificationStatus.QUEUED);
    engine.deliver(notification);
    return notification;
}

public Notification getById(String notificationId) {
    return repository.findById(notificationId)
        .orElseThrow(() -> new NotificationNotFoundException(notificationId));
}

public List<Notification> getUserId(String userId) {
    return repository.findById(userId);
}

```

```

    }

    private void validate(NotificationRequest request) {
        if (request == null) {
            throw new InvalidNotificationRequestException(List.of("request must not be null"));
        }
        Set<ConstraintViolation<NotificationRequest>> violations = validator.validate(request);
        if (!violations.isEmpty()) {
            List<String> messages = violations.stream()
                .map(v -> v.getPropertyPath() + " " + v.getMessage())
                .sorted()
                .toList();
            throw new InvalidNotificationRequestException(messages);
        }
    }

    private NotificationContent composeContent(NotificationRequest request) {
        NotificationContent content = new SimpleNotification(request.title(), request.content());
        content = new TimestampNotificationDecorator(content, ZonedDateTime.now(clock));
        if (StringUtils.hasText(signature)) {
            content = new SignatureNotificationDecorator(content, signature);
        }
        return content;
    }

    private boolean isDeliveryAllowed(Notification notification) {
        return notification.getPriority().overridesUserPreferences()
            || preferenceService.isChannelEnabled(notification.getUserId(), notification.getNotificationType());
    }

    private void changeStatus(Notification notification, NotificationStatus status) {
        notification.transitionTo(status, clock.instant());
        repository.updateStatus(notification);
    }
}

```

12. Persistence

Everything JPA lives here, behind `NotificationRepository`. Service and engine never import a JPA or Spring Data type.

NotificationEntity.java

src/main/java/.../entity/NotificationEntity.java · 71 lines

The database table shape.

- Index on `user_id` for the inbox query; unique constraint on `idempotency_key` for safe deduplication. MySQL allows many `NULL`s in a unique column, so notifications without a key are fine.
- `EnumType.STRING` stores "EMAIL", not `0`. With the default (ordinal), reordering enum values would silently corrupt existing rows.
- `columnDefinition = "varchar(16)"`: found while testing this code. Without it Hibernate creates a MySQL `ENUM(...)` column, and adding `WHATSAPP` later would fail because `ddl-auto: update` never alters an existing column.
- `@Getter` + `@Builder`, not `@Data`: `@Data` generates `equals` / `hashCode` / `setters` that cause subtle bugs with JPA entities.
- The protected no-args constructor exists only because JPA requires one.

```
package org.example.notificationsservice.entity;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.EnumType;
import jakarta.persistence.Enumerated;
import jakarta.persistence.Id;
import jakarta.persistence.Index;
import jakarta.persistence.Table;
import jakarta.persistence.UniqueConstraint;
import lombok.AccessLevel;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import org.example.notificationsservice.enums.NotificationPriority;
import org.example.notificationsservice.enums.NotificationStatus;
import org.example.notificationsservice.enums.NotificationType;

import java.time.Instant;

/** The database table shape. No business Logic here. */
@Entity
@Table(
    name = "notifications",
    indexes = @Index(name = "idx_notifications_user_id", columnList = "user_id"),
    uniqueConstraints = @UniqueConstraint(name = "uk_notifications_idempotency_key", columnNames = "idempotency_key"))
@Getter
@Builder
@NoArgsConstructor(access = AccessLevel.PROTECTED) // JPA needs a no-args constructor
@AllArgsConstructor(access = AccessLevel.PRIVATE) // used by @Builder
public class NotificationEntity {

    @Id
    @Column(name = "notification_id", length = 36)
    private String notificationId;

    @Column(name = "user_id", nullable = false, length = 64)
    private String userId;

    @Column(nullable = false, length = 255)
    private String title;

    // 2000 chars from the request + timestamp and signature added by the decorators
    @Column(nullable = false, length = 4000)
    private String content;

    // EnumType.STRING: store "EMAIL", not 0, so reordering the enum is safe.
    // columnDefinition varchar: otherwise Hibernate generates a MySQL ENUM(...) column, and adding
    // a new value later (e.g. WHATSAPP) fails because ddl-auto=update never alters existing columns.
```

```

    @Enumerated(EnumType.STRING)
    @Column(name = "notification_type", nullable = false, columnDefinition = "varchar(16)")
    private NotificationType notificationType;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, columnDefinition = "varchar(16)")
    private NotificationPriority priority;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, columnDefinition = "varchar(16)")
    private NotificationStatus status;

    @Column(name = "idempotency_key", length = 128)
    private String idempotencyKey;

    @Column(name = "created_at", nullable = false)
    private Instant createdAt;

    @Column(name = "updated_at", nullable = false)
    private Instant updatedAt;
}

```

NotificationRepository.java

src/main/java/.../repository/NotificationRepository.java · 34 lines

The persistence contract (a "port").

- The Javadoc `@throws` notes are part of the contract: callers rely on `DuplicateIdempotencyKeyException` and `NotificationNotFoundException`.
- `updateStatus` writes only two columns, so it can't accidentally overwrite other fields.

```

package org.example.notificationservice.repository;

import org.example.notificationservice.domain.Notification;

import java.util.List;
import java.util.Optional;

/**
 * Persistence abstraction. Service and engine depend on this, never on JPA.
 */
public interface NotificationRepository {

    /**
     * Insert a new notification.
     *
     * @throws org.example.notificationservice.exception.DuplicateIdempotencyKeyException
     *         when another notification already uses the same idempotency key
     */
    void save(Notification notification);

    Optional<Notification> findById(String notificationId);

    Optional<Notification> findByIdempotencyKey(String idempotencyKey);

    /** Newest first. */
    List<Notification> findById(String userId);

    /**
     * Persist only status and updatedAt of an existing notification.
     *
     * @throws org.example.notificationservice.exception.NotificationNotFoundException when it doesn't exist
     */
    void updateStatus(Notification notification);
}

```

Spring Data interface: implementation generated at runtime.

- Package-private (no `public`), so nothing outside `repository.jpa` can bypass the adapter.
- `findByIdOrderByCreatedAtDesc` is a derived query: Spring Data builds the SQL from the method name.
- `@Modifying` + `@Query` for a targeted `UPDATE`. `clearAutomatically = true` stops stale cached entities being returned afterwards. Modifying queries need `@Transactional`.

```
package org.example.notification.service.repository.jpa;

import org.example.notification.service.entity.NotificationEntity;
import org.example.notification.service.enums.NotificationStatus;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Modifying;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.transaction.annotation.Transactional;

import java.time.Instant;
import java.util.List;
import java.util.Optional;

/**
 * Spring Data generates the implementation at runtime.
 * Package-private on purpose: only JpaNotificationRepository may use it.
 */
interface SpringDataNotificationRepository extends JpaRepository<NotificationEntity, String> {

    Optional<NotificationEntity> findByIdempotencyKey(String idempotencyKey);

    boolean existsByIdempotencyKey(String idempotencyKey);

    List<NotificationEntity> findByIdOrderByCreatedAtDesc(String userId);

    @Transactional
    @Modifying(clearAutomatically = true)
    @Query("""
        update NotificationEntity n
        set n.status = :status, n.updatedAt = :updatedAt
        where n.notificationId = :notificationId
        """)
    int updateStatus(@Param("notificationId") String notificationId,
                    @Param("status") NotificationStatus status,
                    @Param("updatedAt") Instant updatedAt);
}
```

JpaNotificationRepository.java

src/main/java/.../repository/jpa/JpaNotificationRepository.java · 68 lines

Adapter implementing our interface with Spring Data.

- `saveAndFlush` forces the `INSERT` immediately, so a unique-constraint violation is thrown inside this method where it can be translated.
- It only translates the error when the key really exists. Any other constraint failure is rethrown unchanged instead of being hidden.
- `updateStatus` checks the affected row count: 0 rows means the notification doesn't exist.

```
package org.example.notification.service.repository.jpa;

import org.example.notification.service.domain.Notification;
import org.example.notification.service.exception.DuplicateIdempotencyKeyException;
import org.example.notification.service.exception.NotificationNotFoundException;
import org.example.notification.service.mapper.NotificationMapper;
import org.example.notification.service.repository.NotificationRepository;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.stereotype.Repository;

import java.util.List;
```

```

import java.util.Optional;

/**
 * Adapter: implements our NotificationRepository using Spring Data JPA.
 * Converts domain <-> entity and translates database errors into our own exceptions.
 */
@Repository
public class JpaNotificationRepository implements NotificationRepository {

    private final SpringDataNotificationRepository springDataRepository;
    private final NotificationMapper mapper;

    public JpaNotificationRepository(SpringDataNotificationRepository springDataRepository, NotificationMapper mapper) {
        this.springDataRepository = springDataRepository;
        this.mapper = mapper;
    }

    @Override
    public void save(Notification notification) {
        try {
            // saveAndFlush so a unique-constraint violation is thrown HERE, not later at commit.
            springDataRepository.saveAndFlush(mapper.toEntity(notification));
        } catch (DataIntegrityViolationException ex) {
            String key = notification.getIdempotencyKey();
            if (key != null && springDataRepository.existsByIdempotencyKey(key)) {
                throw new DuplicateIdempotencyKeyException(key, ex);
            }
            throw ex; // some other constraint failed: don't hide it
        }
    }

    @Override
    public Optional<Notification> findById(String notificationId) {
        return springDataRepository.findById(notificationId).map(mapper::toDomain);
    }

    @Override
    public Optional<Notification> findByIdempotencyKey(String idempotencyKey) {
        return springDataRepository.findByIdempotencyKey(idempotencyKey).map(mapper::toDomain);
    }

    @Override
    public List<Notification> findByUserId(String userId) {
        return springDataRepository.findByIdempotencyKey(userId).stream()
            .map(mapper::toDomain)
            .toList();
    }

    @Override
    public void updateStatus(Notification notification) {
        int updatedRows = springDataRepository.updateStatus(
            notification.getIdempotencyKey(), notification.getStatus(), notification.getIdempotencyKey());
        if (updatedRows == 0) {
            throw new NotificationNotFoundException(notification.getIdempotencyKey());
        }
    }
}

```

NotificationMapper.java

src/main/java/.../mapper/NotificationMapper.java · 40 lines

Copies fields between domain and entity.

- Pure field copying, no decisions. In larger projects MapStruct generates this class for you.

```

package org.example.notificationsservice.mapper;

import org.example.notificationsservice.domain.Notification;
import org.example.notificationsservice.entity.NotificationEntity;
import org.springframework.stereotype.Component;

/** Converts between the domain model and the JPA entity. No logic, just copying fields. */
@Component

```

```
public class NotificationMapper {

    public NotificationEntity toEntity(Notification notification) {
        return NotificationEntity.builder()
            .notificationId(notification.getNotificationId())
            .userId(notification.getUserId())
            .title(notification.getTitle())
            .content(notification.getContent())
            .notificationType(notification.getNotificationType())
            .priority(notification.getPriority())
            .status(notification.getStatus())
            .idempotencyKey(notification.getIdempotencyKey())
            .createdAt(notification.getCreatedAt())
            .updatedAt(notification.getUpdatedAt())
            .build();
    }

    public Notification toDomain(NotificationEntity entity) {
        return Notification.restore()
            .notificationId(entity.getNotificationId())
            .userId(entity.getUserId())
            .title(entity.getTitle())
            .content(entity.getContent())
            .notificationType(entity.getNotificationType())
            .priority(entity.getPriority())
            .status(entity.getStatus())
            .idempotencyKey(entity.getIdempotencyKey())
            .createdAt(entity.getCreatedAt())
            .updatedAt(entity.getUpdatedAt())
            .build();
    }
}
```

13. Web layer and configuration

The thin outer shell: HTTP in, JSON out, and the beans and settings that wire everything together.

NotificationController.java

src/main/java/.../controller/NotificationController.java · 55 lines

REST endpoints under `/api/v1/notifications`.

- `POST` returns `201 Created` with a `Location` header pointing at the new resource.
- No business logic: convert, delegate, convert back.
- Security gap to remember: `GET ?userId=` lets any caller read any user's notifications. Once authentication exists, take the user id from the token, not a query parameter.

```
package org.example.notificationsservice.controller;

import org.example.notificationsservice.domain.Notification;
import org.example.notificationsservice.dto.request.NotificationRequest;
import org.example.notificationsservice.dto.response.NotificationResponse;
import org.example.notificationsservice.service.NotificationService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.servlet.support.ServletUriComponentsBuilder;

import java.net.URI;
import java.util.List;

/**
 * Thin HTTP Layer: converts JSON <-> DTOs and delegates. No @Valid here because
 * NotificationService validates, so callers that skip HTTP get the same checks.
 */
@RestController
@RequestMapping("/api/v1/notifications")
public class NotificationController {

    private final NotificationService notificationService;

    public NotificationController(NotificationService notificationService) {
        this.notificationService = notificationService;
    }

    @PostMapping
    public ResponseEntity<NotificationResponse> send(@RequestBody NotificationRequest request) {
        Notification notification = notificationService.send(request);
        URI location = ServletUriComponentsBuilder.fromCurrentRequest()
            .path("/{id}")
            .buildAndExpand(notification.getNotificationId())
            .toUri();
        return ResponseEntity.created(location).body(NotificationResponse.from(notification));
    }

    @GetMapping("/{notificationId}")
    public NotificationResponse getById(@PathVariable String notificationId) {
        return NotificationResponse.from(notificationService.getById(notificationId));
    }

    @GetMapping(params = "userId")
    public List<NotificationResponse> getByUserId(@RequestParam String userId) {
        return notificationService.getByUserId(userId).stream()
            .map(NotificationResponse::from)
            .toList();
    }
}
```

Beans that need configuration values.

- `Clock` as a bean: production uses the real clock, tests use `Clock.fixed(...)`.
- `@Value("${notification.retry.max-attempts:3}")`: the part after `:` is the default when the property is missing.

```
package org.example.notification.service.config;

import org.example.notification.service.retry.ExponentialBackoffRetryPolicy;
import org.example.notification.service.retry.RetryPolicy;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import java.time.Clock;
import java.time.Duration;
import java.time.ZoneId;

@Configuration
public class NotificationConfig {

    /** Injected everywhere instead of calling Instant.now(), so tests can use a fixed clock. */
    @Bean
    public Clock clock(@Value("${notification.content.time-zone:UTC}") String timeZone) {
        return Clock.system(ZoneId.of(timeZone));
    }

    @Bean
    public RetryPolicy retryPolicy(@Value("${notification.retry.max-attempts:3}") int maxAttempts,
                                  @Value("${notification.retry.initial-delay:500ms}") Duration initialDelay,
                                  @Value("${notification.retry.multiplier:2.0}") double multiplier,
                                  @Value("${notification.retry.max-delay:5s}") Duration maxDelay) {
        return new ExponentialBackoffRetryPolicy(maxAttempts, initialDelay, multiplier, maxDelay);
    }
}
```

14. Tests

Unit tests for each building block, a real-database test for persistence, and one test that starts the whole application. Tests use a fixed `clock` and zero retry delays, so they are fast and give the same result every run.

NotificationTest.java

src/test/java/.../domain/NotificationTest.java · 63 lines

Lifecycle rules of the domain object.

- Checks that an illegal move throws **and** that the status didn't change: a failed operation must leave the object untouched.

```
package org.example.notification.service.domain;

import org.example.notification.service.enums.NotificationPriority;
import org.example.notification.service.enums.NotificationStatus;
import org.example.notification.service.enums.NotificationType;
import org.junit.jupiter.api.Test;

import java.time.Instant;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;

class NotificationTest {

    private static final Instant NOW = Instant.parse("2026-09-16T10:15:32Z");

    private static Notification newNotification() {
        return Notification.create("user-1", "Money debited", "INR 5,000 debited",
            NotificationType.SMS, NotificationPriority.HIGH, "txn-1:debit", NOW);
    }

    @Test
    void createStartsInCreatedStatusWithGeneratedId() {
        Notification notification = newNotification();

        assertThat(notification.getNotificationId()).isNotBlank();
        assertThat(notification.getStatus()).isEqualTo(NotificationStatus.CREATED);
        assertThat(notification.getCreatedAt()).isEqualTo(NOW);
        assertThat(notification.getUpdatedAt()).isEqualTo(NOW);
    }

    @Test
    void allowedTransitionUpdatesStatusAndTimestamp() {
        Notification notification = newNotification();
        Instant later = NOW.plusSeconds(5);

        notification.transitionTo(NotificationStatus.QUEUED, later);

        assertThat(notification.getStatus()).isEqualTo(NotificationStatus.QUEUED);
        assertThat(notification.getUpdatedAt()).isEqualTo(later);
        assertThat(notification.getCreatedAt()).isEqualTo(NOW);
    }

    @Test
    void illegalTransitionIsRejected() {
        Notification notification = newNotification();

        assertThatThrownBy(() -> notification.transitionTo(NotificationStatus.SENT, NOW))
            .isInstanceOf(IllegalStateException.class)
            .hasMessageContaining("CREATED to SENT");
        assertThat(notification.getStatus()).isEqualTo(NotificationStatus.CREATED);
    }

    @Test
    void terminalStatusesCannotMoveAnywhere() {
        for (NotificationStatus terminal : new NotificationStatus[]{
            NotificationStatus.SENT, NotificationStatus.FAILED, NotificationStatus.CANCELLED}) {
            for (NotificationStatus next : NotificationStatus.values()) {
                assertThat(terminal.canTransitionTo(next)).as("%s -> %s", terminal, next).isFalse();
            }
        }
    }
}
```

```

    }
}
}
}

```

NotificationDecoratorTest.java

src/test/java/.../decorator/NotificationDecoratorTest.java · 69 lines

Each decorator alone, then composed.

- Java text blocks (`"""`) make the expected multi-line body readable.
- `wrappingOrderDecidesOutputOrder` proves the order of wrapping controls the output.

```

package org.example.notificationsservice.decorator;

import org.junit.jupiter.api.Test;

import java.time.ZoneId;
import java.time.ZonedDateTime;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;

class NotificationDecoratorTest {

    private static final ZonedDateTime GENERATED_AT =
        ZonedDateTime.of(2026, 9, 16, 10, 15, 32, 0, ZoneId.of("UTC"));

    private final NotificationContent simple =
        new SimpleNotification("Money debited", "INR 5,000.00 debited from A/c XX1234.");

    @Test
    void simpleNotificationReturnsOriginalText() {
        assertThat(simple.getTitle()).isEqualTo("Money debited");
        assertThat(simple.getBody()).isEqualTo("INR 5,000.00 debited from A/c XX1234.");
    }

    @Test
    void timestampDecoratorAppendsFormattedTime() {
        NotificationContent content = new TimestampNotificationDecorator(simple, GENERATED_AT);

        assertThat(content.getBody())
            .isEqualTo("INR 5,000.00 debited from A/c XX1234.\n\nTime: 16 Sep 2026, 10:15:32 UTC");
    }

    @Test
    void signatureDecoratorAppendsSignature() {
        NotificationContent content = new SignatureNotificationDecorator(simple, "- Enterprise Digital Bank");

        assertThat(content.getBody())
            .isEqualTo("INR 5,000.00 debited from A/c XX1234.\n\n- Enterprise Digital Bank");
    }

    @Test
    void decoratorsComposeByWrappingEachOther() {
        NotificationContent content = simple;
        content = new TimestampNotificationDecorator(content, GENERATED_AT);
        content = new SignatureNotificationDecorator(content, "- Enterprise Digital Bank");

        assertThat(content.getTitle()).isEqualTo("Money debited");
        assertThat(content.getBody()).isEqualTo("""
            INR 5,000.00 debited from A/c XX1234.

            Time: 16 Sep 2026, 10:15:32 UTC

            - Enterprise Digital Bank""");
    }

    @Test
    void wrappingOrderDecidesOutputOrder() {
        NotificationContent content =
            new TimestampNotificationDecorator(new SignatureNotificationDecorator(simple, "- Bank"), GENERATED_AT);
    }
}

```

```

        assertThat(content.getBody()).endsWith("- Bank\n\nTime: 16 Sep 2026, 10:15:32 UTC");
    }

    @Test
    void decoratorRequiresSomethingToWrap() {
        assertThatThrownBy(() -> new SignatureNotificationDecorator(null, "- Bank"))
            .isInstanceOf(NullPointerException.class);
    }
}

```

ExponentialBackoffRetryPolicyTest.java

src/test/java/.../retry/ExponentialBackoffRetryPolicyTest.java · 54 lines

Delay maths, retry classification, config validation.

```

package org.example.notificationsservice.retry;

import org.example.notificationsservice.exception.NotificationDeliveryException;
import org.junit.jupiter.api.Test;

import java.time.Duration;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;

class ExponentialBackoffRetryPolicyTest {

    private final ExponentialBackoffRetryPolicy policy =
        new ExponentialBackoffRetryPolicy(5, Duration.ofMillis(100), 2.0, Duration.ofSeconds(1));

    @Test
    void delayDoublesAfterEachFailureUntilCapped() {
        assertThat(policy.delayAfterAttempt(1)).isEqualTo(Duration.ofMillis(100));
        assertThat(policy.delayAfterAttempt(2)).isEqualTo(Duration.ofMillis(200));
        assertThat(policy.delayAfterAttempt(3)).isEqualTo(Duration.ofMillis(400));
        assertThat(policy.delayAfterAttempt(4)).isEqualTo(Duration.ofMillis(800));
        assertThat(policy.delayAfterAttempt(5)).isEqualTo(Duration.ofSeconds(1));
        assertThat(policy.delayAfterAttempt(5000)).isEqualTo(Duration.ofSeconds(1));
    }

    @Test
    void onlyRetryableDeliveryFailuresAreRetried() {
        assertThat(policy.isRetryable(NotificationDeliveryException.retryable("gateway timeout", null))).isTrue();
        assertThat(policy.isRetryable(NotificationDeliveryException.permanent("invalid number", null))).isFalse();
        assertThat(policy.isRetryable(new IllegalStateException("bug"))).isFalse();
    }

    @Test
    void exposesMaxAttempts() {
        assertThat(policy.maxAttempts()).isEqualTo(5);
    }

    @Test
    void rejectsInvalidConfiguration() {
        assertThatThrownBy(() -> new ExponentialBackoffRetryPolicy(0, Duration.ZERO, 2.0, Duration.ZERO))
            .isInstanceOf(IllegalArgumentException.class);
        assertThatThrownBy(() -> new ExponentialBackoffRetryPolicy(3, Duration.ofMillis(-1), 2.0, Duration.ZERO))
            .isInstanceOf(IllegalArgumentException.class);
        assertThatThrownBy(() -> new ExponentialBackoffRetryPolicy(3, Duration.ZERO, 0.5, Duration.ZERO))
            .isInstanceOf(IllegalArgumentException.class);
        assertThatThrownBy(() -> new ExponentialBackoffRetryPolicy(3, Duration.ofSeconds(2), 2.0, Duration.ofSeconds(1)))
            .isInstanceOf(IllegalArgumentException.class);
    }

    @Test
    void rejectsAttemptNumbersBelowOne() {
        assertThatThrownBy(() -> policy.delayAfterAttempt(0)).isInstanceOf(IllegalArgumentException.class);
    }
}

```

Lookup, missing type, duplicate registration.

- An anonymous class creates a second SMS channel just for the duplicate test.

```
package org.example.notificationsservice.factory;

import org.example.notificationsservice.domain.Notification;
import org.example.notificationsservice.enums.NotificationType;
import org.example.notificationsservice.exception.UnsupportedNotificationChannelException;
import org.example.notificationsservice.strategy.EmailNotificationChannel;
import org.example.notificationsservice.strategy.InAppNotificationChannel;
import org.example.notificationsservice.strategy.NotificationChannel;
import org.example.notificationsservice.strategy.PushNotificationChannel;
import org.example.notificationsservice.strategy.SmsNotificationChannel;
import org.junit.jupiter.api.Test;

import java.util.List;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;

class NotificationChannelResolverTest {

    @Test
    void resolvesTheChannelThatSupportsEachType() {
        EmailNotificationChannel email = new EmailNotificationChannel();
        SmsNotificationChannel sms = new SmsNotificationChannel();
        PushNotificationChannel push = new PushNotificationChannel();
        InAppNotificationChannel inApp = new InAppNotificationChannel();
        NotificationChannelResolver resolver = new NotificationChannelResolver(List.of(email, sms, push, inApp));

        assertThat(resolver.resolve(NotificationType.EMAIL)).isSameAs(email);
        assertThat(resolver.resolve(NotificationType.SMS)).isSameAs(sms);
        assertThat(resolver.resolve(NotificationType.PUSH)).isSameAs(push);
        assertThat(resolver.resolve(NotificationType.IN_APP)).isSameAs(inApp);
    }

    @Test
    void throwsWhenNoChannelIsRegisteredForType() {
        NotificationChannelResolver resolver = new NotificationChannelResolver(List.of(new SmsNotificationChannel()));

        assertThatThrownBy(() -> resolver.resolve(NotificationType.EMAIL))
            .isInstanceOf(UnsupportedNotificationChannelException.class)
            .hasMessageContaining("EMAIL");
    }

    @Test
    void failsFastWhenTwoChannelsClaimTheSameType() {
        NotificationChannel secondSms = new NotificationChannel() {
            @Override
            public NotificationType supportedType() {
                return NotificationType.SMS;
            }

            @Override
            public void send(Notification notification) {
            }
        };

        assertThatThrownBy(() -> new NotificationChannelResolver(List.of(new SmsNotificationChannel(), secondSms)))
            .isInstanceOf(IllegalStateException.class)
            .hasMessageContaining("SMS");
    }
}
```

Fan-out order, failure isolation, subscribe/unsubscribe, metrics.

- Listeners are lambdas: possible because NotificationEventListener has one method.

- The ERROR line this test prints in the build log is expected: it's the deliberately broken listener.

```
package org.example.notification.service.observer;

import org.example.notification.service.domain.Notification;
import org.example.notification.service.enums.NotificationPriority;
import org.example.notification.service.enums.NotificationType;
import org.junit.jupiter.api.Test;

import java.time.Instant;
import java.util.ArrayList;
import java.util.List;

import static org.assertj.core.api.Assertions.assertThat;

class NotificationEventPublisherTest {

    private static final Instant NOW = Instant.parse("2026-09-16T10:15:32Z");

    private final NotificationEvent sentEvent = NotificationEvent.of(
        NotificationEventType.NOTIFICATION_SENT,
        Notification.create("user-1", "Title", "Body", NotificationType.EMAIL, NotificationPriority.LOW, null, NOW),
        NOW);

    @Test
    void notifiesEveryRegisteredListenerInOrder() {
        List<String> calls = new ArrayList<>();
        NotificationEventPublisher publisher = new NotificationEventPublisher(List.of(
            event -> calls.add("audit:" + event.eventType()),
            event -> calls.add("logging:" + event.eventType()),
            event -> calls.add("metrics:" + event.eventType())));

        publisher.publish(sentEvent);

        assertThat(calls).containsExactly(
            "audit:NOTIFICATION_SENT", "logging:NOTIFICATION_SENT", "metrics:NOTIFICATION_SENT");
    }

    @Test
    void failingListenerDoesNotStopTheOthers() {
        List<String> calls = new ArrayList<>();
        NotificationEventPublisher publisher = new NotificationEventPublisher(List.of(
            event -> calls.add("first"),
            event -> {
                throw new IllegalStateException("listener bug");
            },
            event -> calls.add("third")));

        publisher.publish(sentEvent);

        assertThat(calls).containsExactly("first", "third");
    }

    @Test
    void subscribeAndUnsubscribeChangeWhoIsNotified() {
        List<NotificationEvent> received = new ArrayList<>();
        NotificationEventListener listener = received::add;
        NotificationEventPublisher publisher = new NotificationEventPublisher(List.of());

        publisher.subscribe(listener);
        publisher.subscribe(listener); // ignored: already subscribed
        publisher.publish(sentEvent);
        publisher.unsubscribe(listener);
        publisher.publish(sentEvent);

        assertThat(received).containsExactly(sentEvent);
    }

    @Test
    void metricsListenerCountsEventsByType() {
        NotificationMetricsListener metrics = new NotificationMetricsListener();
        NotificationEventPublisher publisher = new NotificationEventPublisher(List.of(metrics));

        publisher.publish(sentEvent);
    }
}
```

```

        publisher.publish(sentEvent);

        assertThat(metrics.count(NotificationEventType.NOTIFICATION_SENT)).isEqualTo(2);
        assertThat(metrics.count(NotificationEventType.NOTIFICATION_FAILED)).isZero();
    }
}

```

NotificationEngineTest.java

src/test/java/.../engine/NotificationEngineTest.java · 184 lines

Every delivery path: success, retry, permanent failure, exhaustion, bug, missing channel.

- `ScriptedSmsChannel` is a hand-written **fake**, not a Mockito mock: "fail twice then succeed" is much clearer as real code.
- The repository **is** a mock, with `doAnswer` recording the status at each call. Needed because the same `Notification` object keeps changing; a plain `verify` would only see its final state.
- A zero-delay retry policy keeps the tests instant.

```

package org.example.notificationengine;

import org.example.notificationengine.domain.Notification;
import org.example.notificationengine.enums.NotificationPriority;
import org.example.notificationengine.enums.NotificationStatus;
import org.example.notificationengine.enums.NotificationType;
import org.example.notificationengine.exception.NotificationDeliveryException;
import org.example.notificationengine.factory.NotificationChannelResolver;
import org.example.notificationengine.observer.NotificationEvent;
import org.example.notificationengine.observer.NotificationEventPublisher;
import org.example.notificationengine.repository.NotificationRepository;
import org.example.notificationengine.retry.ExponentialBackoffRetryPolicy;
import org.example.notificationengine.strategy.NotificationChannel;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import java.time.Clock;
import java.time.Duration;
import java.time.Instant;
import java.time.ZoneId;
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Deque;
import java.util.List;

import static org.assertj.core.api.Assertions.assertThat;
import static org.example.notificationengine.enums.NotificationStatus.FAILED;
import static org.example.notificationengine.enums.NotificationStatus.PROCESSING;
import static org.example.notificationengine.enums.NotificationStatus.RETRYING;
import static org.example.notificationengine.enums.NotificationStatus.SENT;
import static org.example.notificationengine.observer.NotificationEventType.NOTIFICATION_FAILED;
import static org.example.notificationengine.observer.NotificationEventType.NOTIFICATION_RETRYING;
import static org.example.notificationengine.observer.NotificationEventType.NOTIFICATION_SENT;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.doAnswer;
import static org.mockito.Mockito.mock;

class NotificationEngineTest {

    private static final Instant NOW = Instant.parse("2026-09-16T10:15:32Z");
    private static final Clock CLOCK = Clock.fixed(NOW, ZoneId.of("UTC"));

    private final NotificationRepository repository = mock(NotificationRepository.class);
    private final List<NotificationStatus> persistedStatuses = new ArrayList<>();
    private final List<NotificationEvent> events = new ArrayList<>();
    private final NotificationEventPublisher publisher = new NotificationEventPublisher(List.of(events::add));

    @BeforeEach
    void recordEveryStatusThatIsPersisted() {
        // The same Notification object is mutated, so capture the status at the moment of each call.
        doAnswer(invocation -> {
            persistedStatuses.add(invocation.<Notification>getArgument(0).getStatus());
            return null;
        }).when(repository).updateStatus(any());
    }
}

```

```

}

@Test
void sendsSuccessfullyOnFirstAttempt() {
    ScriptedSmsChannel channel = new ScriptedSmsChannel();
    Notification notification = queuedSms();

    engineWith(channel, 3).deliver(notification);

    assertThat(notification.getStatus()).isEqualTo(SENT);
    assertThat(persistedStatuses).containsExactly(PROCESSING, SENT);
    assertThat(events).extracting(NotificationEvent::eventType).containsExactly(NOTIFICATION_SENT);
    assertThat(channel.calls).isEqualTo(1);
}

@Test
void retriesRetryableFailureAndThenSucceeds() {
    ScriptedSmsChannel channel = new ScriptedSmsChannel(
        NotificationDeliveryException.retryable("gateway timeout", null));
    Notification notification = queuedSms();

    engineWith(channel, 3).deliver(notification);

    assertThat(notification.getStatus()).isEqualTo(SENT);
    assertThat(persistedStatuses).containsExactly(PROCESSING, RETRYING, PROCESSING, SENT);
    assertThat(events).extracting(NotificationEvent::eventType)
        .containsExactly(NOTIFICATION_RETRYING, NOTIFICATION_SENT);
    assertThat(channel.calls).isEqualTo(2);
}

@Test
void failsImmediatelyOnPermanentFailure() {
    ScriptedSmsChannel channel = new ScriptedSmsChannel(
        NotificationDeliveryException.permanent("invalid phone number", null));
    Notification notification = queuedSms();

    engineWith(channel, 3).deliver(notification);

    assertThat(notification.getStatus()).isEqualTo(FAILED);
    assertThat(persistedStatuses).containsExactly(PROCESSING, FAILED);
    assertThat(events).singleElement().satisfies(event -> {
        assertThat(event.eventType()).isEqualTo(NOTIFICATION_FAILED);
        assertThat(event.reason()).isEqualTo("invalid phone number");
    });
    assertThat(channel.calls).isEqualTo(1);
}

@Test
void givesUpAfterMaxAttempts() {
    ScriptedSmsChannel channel = new ScriptedSmsChannel(
        NotificationDeliveryException.retryable("timeout 1", null),
        NotificationDeliveryException.retryable("timeout 2", null),
        NotificationDeliveryException.retryable("timeout 3", null));
    Notification notification = queuedSms();

    engineWith(channel, 3).deliver(notification);

    assertThat(notification.getStatus()).isEqualTo(FAILED);
    assertThat(persistedStatuses).containsExactly(PROCESSING, RETRYING, PROCESSING, RETRYING, PROCESSING, FAILED);
    assertThat(events).extracting(NotificationEvent::eventType)
        .containsExactly(NOTIFICATION_RETRYING, NOTIFICATION_RETRYING, NOTIFICATION_FAILED);
    assertThat(channel.calls).isEqualTo(3);
}

@Test
void doesNotRetryUnexpectedExceptions() {
    ScriptedSmsChannel channel = new ScriptedSmsChannel(new IllegalStateException("bug in channel"));
    Notification notification = queuedSms();

    engineWith(channel, 3).deliver(notification);

    assertThat(notification.getStatus()).isEqualTo(FAILED);
    assertThat(channel.calls).isEqualTo(1);
}

```

```

@Test
void failsWhenNoChannelExistsForTheType() {
    NotificationEngine engine = new NotificationEngine(new NotificationChannelResolver(List.of()),
        repository, publisher, zeroDelayPolicy(3), CLOCK);
    Notification notification = queuedSms();

    engine.deliver(notification);

    assertThat(notification.getStatus()).isEqualTo(FAILED);
    assertThat(persistedStatuses).containsExactly(FAILED);
    assertThat(events).extracting(NotificationEvent::eventType).containsExactly(NOTIFICATION_FAILED);
}

private NotificationEngine engineWith(NotificationChannel channel, int maxAttempts) {
    return new NotificationEngine(new NotificationChannelResolver(List.of(channel)),
        repository, publisher, zeroDelayPolicy(maxAttempts), CLOCK);
}

/** Zero delays so tests don't sleep. */
private static ExponentialBackoffRetryPolicy zeroDelayPolicy(int maxAttempts) {
    return new ExponentialBackoffRetryPolicy(maxAttempts, Duration.ZERO, 1.0, Duration.ZERO);
}

private static Notification queuedSms() {
    Notification notification = Notification.create("user-1", "Money debited", "INR 5,000 debited",
        NotificationType.SMS, NotificationPriority.HIGH, null, NOW);
    notification.transitionTo(NotificationStatus.QUEUED, NOW);
    return notification;
}

/** Fake SMS channel: throws the given exceptions in order, then succeeds. */
private static final class ScriptedSmsChannel implements NotificationChannel {

    private final Deque<RuntimeException> failures;
    private int calls;

    ScriptedSmsChannel(RuntimeException... failures) {
        this.failures = new ArrayDeque<>(Arrays.asList(failures));
    }

    @Override
    public NotificationType supportedType() {
        return NotificationType.SMS;
    }

    @Override
    public void send(Notification notification) {
        calls++;
        if (!failures.isEmpty()) {
            throw failures.poll();
        }
    }
}
}

```

NotificationServiceTest.java

src/test/java/.../service/NotificationServiceTest.java · 180 lines

Orchestration: happy path, idempotency, race, validation, preferences.

- Mocks only for the boundaries (repository, engine). Real publisher, preferences and validator, because they're simple and fast.
- The fixed clock is why the test can assert the exact decorated body, timestamp included.
- `statusWhenEngineCalled` proves the service marks the notification `QUEUED` *before* handing it over.

```

package org.example.notificationservice.service;

import jakarta.validation.Validation;
import jakarta.validation.ValidatorFactory;
import org.example.notificationservice.domain.Notification;
import org.example.notificationservice.dto.request.NotificationRequest;
import org.example.notificationservice.engine.NotificationEngine;

```

```

import org.example.notification.service.enums.NotificationPriority;
import org.example.notification.service.enums.NotificationStatus;
import org.example.notification.service.enums.NotificationType;
import org.example.notification.service.exception.DuplicateIdempotencyKeyException;
import org.example.notification.service.exception.InvalidNotificationRequestException;
import org.example.notification.service.exception.NotificationNotFoundException;
import org.example.notification.service.observer.NotificationEvent;
import org.example.notification.service.observer.NotificationEventPublisher;
import org.example.notification.service.preference.InMemoryNotificationPreferenceService;
import org.example.notification.service.repository.NotificationRepository;
import org.junit.jupiter.api.AfterAll;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import java.time.Clock;
import java.time.Instant;
import java.time.ZoneId;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;
import static org.example.notification.service.observer.NotificationEventType.NOTIFICATION_CANCELLED;
import static org.example.notification.service.observer.NotificationEventType.NOTIFICATION_CREATED;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.ArgumentMatchers.anyString;
import static org.mockito.Mockito.doAnswer;
import static org.mockito.Mockito.doThrow;
import static org.mockito.Mockito.mock;
import static org.mockito.Mockito.never;
import static org.mockito.Mockito.verify;
import static org.mockito.Mockito.verifyNoInteractions;
import static org.mockito.Mockito.when;

class NotificationServiceTest {

    private static final Instant NOW = Instant.parse("2026-09-16T10:15:32Z");
    private static final Clock CLOCK = Clock.fixed(NOW, ZoneId.of("UTC"));

    private static ValidatorFactory validatorFactory;

    private final NotificationRepository repository = mock(NotificationRepository.class);
    private final NotificationEngine engine = mock(NotificationEngine.class);
    private final InMemoryNotificationPreferenceService preferences = new InMemoryNotificationPreferenceService();
    private final List<NotificationEvent> events = new ArrayList<>();
    private NotificationService service;

    @BeforeAll
    static void createValidator() {
        validatorFactory = Validation.buildDefaultValidatorFactory();
    }

    @AfterAll
    static void closeValidator() {
        validatorFactory.close();
    }

    @BeforeEach
    void setUp() {
        NotificationEventPublisher publisher = new NotificationEventPublisher(List.of(events::add));
        service = new NotificationService(repository, engine, publisher, preferences,
            validatorFactory.getValidator(), CLOCK, "- Enterprise Digital Bank");
    }

    @Test
    void createsDecoratesPersistsAndDeliversNewNotification() {
        when(repository.findByIdempotencyKey("txn-1:debit")).thenReturn(Optional.empty());
        List<NotificationStatus> statusWhenEngineCalled = new ArrayList<>();
        doAnswer(invocation -> statusWhenEngineCalled.add(invocation.<Notification>getArgument(0).getStatus()))
            .when(engine).deliver(any());

        Notification result = service.send(request(NotificationType.SMS, NotificationPriority.HIGH, "txn-1:debit"));
    }

```

```

        verify(repository).save(result);
        verify(engine).deliver(result);
        assertThat(result.getIdempotencyKey()).isEqualTo("txn-1:debit");
        assertThat(result.getContent()).isEqualTo("""
            INR 5,000 debited from A/c XX1234.

            Time: 16 Sep 2026, 10:15:32 UTC

            - Enterprise Digital Bank""");
        assertThat(statusWhenEngineCalled).containsExactly(NotificationStatus.QUEUED);
        assertThat(events).extracting(NotificationEvent::eventType).containsExactly(NOTIFICATION_CREATED);
    }

    @Test
    void returnsExistingNotificationWhenIdempotencyKeyWasAlreadyUsed() {
        Notification existing = existingNotification("txn-1:debit");
        when(repository.findByIdempotencyKey("txn-1:debit")).thenReturn(Optional.of(existing));

        Notification result = service.send(request(NotificationType.SMS, NotificationPriority.HIGH, "txn-1:debit"));

        assertThat(result).isSameAs(existing);
        verify(repository, never()).save(any());
        verifyNoInteractions(engine);
        assertThat(events).isEmpty();
    }

    @Test
    void returnsWinnerWhenConcurrentRequestInsertedTheSameKeyFirst() {
        Notification winner = existingNotification("txn-1:debit");
        when(repository.findByIdempotencyKey("txn-1:debit")).thenReturn(Optional.empty(), Optional.of(winner));
        doThrow(new DuplicateIdempotencyKeyException("txn-1:debit", null)).when(repository).save(any());

        Notification result = service.send(request(NotificationType.SMS, NotificationPriority.HIGH, "txn-1:debit"));

        assertThat(result).isSameAs(winner);
        verifyNoInteractions(engine);
        assertThat(events).isEmpty();
    }

    @Test
    void rejectsInvalidRequestWithoutSideEffects() {
        NotificationRequest invalid = new NotificationRequest(" ", "Title", "Body", null, NotificationPriority.LOW, null);

        assertThatThrownBy(() -> service.send(invalid))
            .isInstanceOfSatisfying(InvalidNotificationRequestException.class, ex -> {
                assertThat(ex.getViolations()).anyMatch(v -> v.startsWith("userId "));
                assertThat(ex.getViolations()).anyMatch(v -> v.startsWith("notificationType "));
            });
        verifyNoInteractions(repository, engine);
    }

    @Test
    void cancelsWhenUserDisabledTheChannel() {
        preferences.setChannelEnabled("user-1", NotificationType.SMS, false);

        Notification result = service.send(request(NotificationType.SMS, NotificationPriority.HIGH, null));

        assertThat(result.getStatus()).isEqualTo(NotificationStatus.CANCELLED);
        verify(repository).updateStatus(result);
        verifyNoInteractions(engine);
        assertThat(events).extracting(NotificationEvent::eventType)
            .containsExactly(NOTIFICATION_CREATED, NOTIFICATION_CANCELLED);
    }

    @Test
    void criticalNotificationIgnoresDisabledChannel() {
        preferences.setChannelEnabled("user-1", NotificationType.SMS, false);

        Notification result = service.send(request(NotificationType.SMS, NotificationPriority.CRITICAL, null));

        verify(engine).deliver(result);
        assertThat(result.getStatus()).isEqualTo(NotificationStatus.QUEUED);
    }

```

```

@Test
void blankIdempotencyKeyIsTreatedAsNoKey() {
    Notification result = service.send(request(NotificationType.EMAIL, NotificationPriority.LOW, "  "));

    assertThat(result.getIdempotencyKey()).isNull();
    verify(repository, never()).findByIdempotencyKey(anyString());
}

@Test
void getIdByIdThrowsWhenNotificationDoesNotExist() {
    when(repository.findById("missing")).thenReturn(Optional.empty());

    assertThatThrownBy(() -> service.getIdById("missing")).isInstanceOf(NotificationNotFoundException.class);
}

private static NotificationRequest request(NotificationType type, NotificationPriority priority, String key) {
    return new NotificationRequest("user-1", "Money debited", "INR 5,000 debited from A/c XX1234.", type, priority, key);
}

private static Notification existingNotification(String key) {
    return Notification.create("user-1", "Money debited", "earlier body",
        NotificationType.SMS, NotificationPriority.HIGH, key, NOW.minusSeconds(60));
}
}

```

JpaNotificationRepositoryTest.java

src/test/java/.../repository/jpa/JpaNotificationRepositoryTest.java · 123 lines

Real SQL against in-memory H2.

- `@DataJpaTest` starts only the JPA part of Spring: fast, with no web layer.
- `Propagation.NOT_SUPPORTED` disables the default test-wide transaction. Otherwise a unique-constraint failure would break the shared session and the test couldn't behave like production.
- `usingRecursiveComparison()` compares every field of the saved and loaded notification in one assertion.

```

package org.example.notification.service.repository.jpa;

import org.example.notification.service.domain.Notification;
import org.example.notification.service.enums.NotificationPriority;
import org.example.notification.service.enums.NotificationStatus;
import org.example.notification.service.enums.NotificationType;
import org.example.notification.service.exception.DuplicateIdempotencyKeyException;
import org.example.notification.service.exception.NotificationNotFoundException;
import org.example.notification.service.mapper.NotificationMapper;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.data.jpa.test.autoconfigure.DataJpaTest;
import org.springframework.context.annotation.Import;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;

import java.time.Instant;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;

/**
 * Runs against an in-memory H2 database.
 * NOT_SUPPORTED: no test-wide transaction, so each call commits like it does in production
 * (needed to see the unique-constraint violation the way the service will).
 */
@DataJpaTest
@Import({JpaNotificationRepository.class, NotificationMapper.class})
@Transactional(propagation = Propagation.NOT_SUPPORTED)
class JpaNotificationRepositoryTest {

    private static final Instant NOW = Instant.parse("2026-09-16T10:15:32Z");

    @Autowired
    private JpaNotificationRepository repository;

```

```

@Autowired
private SpringDataNotificationRepository springDataRepository;

@AfterEach
void deleteAll() {
    springDataRepository.deleteAll();
}

@Test
void savesAndLoadsEveryField() {
    Notification notification = notification("user-1", "txn-1:debit", NOW);

    repository.save(notification);

    Notification loaded = repository.findById(notification.getId()).orElseThrow();
    assertThat(loaded).usingRecursiveComparison().isEqualTo(notification);
}

@Test
void findsByIdempotencyKey() {
    Notification notification = notification("user-1", "txn-1:debit", NOW);
    repository.save(notification);

    assertThat(repository.findByIdempotencyKey("txn-1:debit"))
        .get().extracting(Notification::getId).isEqualTo(notification.getId());
    assertThat(repository.findByIdempotencyKey("unknown")).isEmpty();
}

@Test
void findsNotificationsOfAUserNewestFirst() {
    Notification older = notification("user-1", null, NOW);
    Notification newer = notification("user-1", null, NOW.plusSeconds(60));
    Notification otherUser = notification("user-2", null, NOW);
    repository.save(older);
    repository.save(newer);
    repository.save(otherUser);

    assertThat(repository.findById("user-1"))
        .extracting(Notification::getId)
        .containsExactly(newer.getId(), older.getId());
}

@Test
void updatesOnlyStatusAndUpdatedAt() {
    Notification notification = notification("user-1", null, NOW);
    repository.save(notification);
    Instant later = NOW.plusSeconds(3);
    notification.transitionTo(NotificationStatus.QUEUED, later);

    repository.updateStatus(notification);

    Notification loaded = repository.findById(notification.getId()).orElseThrow();
    assertThat(loaded.getStatus()).isEqualTo(NotificationStatus.QUEUED);
    assertThat(loaded.getUpdatedAt()).isEqualTo(later);
    assertThat(loaded.getCreatedAt()).isEqualTo(NOW);
}

@Test
void updatingUnknownNotificationThrows() {
    Notification neverSaved = notification("user-1", null, NOW);

    assertThatThrownBy(() -> repository.updateStatus(neverSaved))
        .isInstanceOf(NotificationNotFoundException.class);
}

@Test
void secondNotificationWithSameIdempotencyKeyIsRejected() {
    repository.save(notification("user-1", "txn-1:debit", NOW));

    assertThatThrownBy(() -> repository.save(notification("user-1", "txn-1:debit", NOW)))
        .isInstanceOf(DuplicateIdempotencyKeyException.class);
}

```

```

@Test
void manyNotificationsMayHaveNoIdempotencyKey() {
    repository.save(notification("user-1", null, NOW));
    repository.save(notification("user-1", null, NOW));

    assertThat(repository.findById("user-1")).hasSize(2);
}

private static Notification notification(String userId, String idempotencyKey, Instant createdAt) {
    return Notification.create(userId, "Money debited", "INR 5,000 debited",
        NotificationType.SMS, NotificationPriority.HIGH, idempotencyKey, createdAt);
}
}

```

NotificationApiIntegrationTest.java

src/test/java/.../NotificationApiIntegrationTest.java · 93 lines

The whole application over HTTP.

- `@SpringBootTest` starts everything; `@AutoConfigureMockMvc` sends HTTP requests without a real server.
- Properties swap MySQL for H2 and remove retry delays, so the test needs no database installed.
- If a bean is mis-wired (missing `@Component`, wrong constructor), this is the test that catches it.

```

package org.example.notificationservice;

import com.jayway.jsonpath.JsonPath;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.webMvc.test.autoconfigure.AutoConfigureMockMvc;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.MvcResult;

import static org.hamcrest.Matchers.hasItem;
import static org.hamcrest.Matchers.startsWith;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.header;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

/**
 * Starts the whole application (H2 instead of MySQL) and calls the real HTTP endpoints.
 * Proves all the beans are wired together correctly.
 */
@SpringBootTest(properties = {
    "spring.datasource.url=jdbc:h2:mem:notification-it;DB_CLOSE_DELAY=-1",
    "spring.datasource.username=sa",
    "spring.datasource.password=",
    "spring.jpa.hibernate.ddl-auto=create-drop",
    "notification.retry.initial-delay=0ms",
    "notification.retry.max-delay=0ms"
})
@AutoConfigureMockMvc
class NotificationApiIntegrationTest {

    @Autowired
    private MockMvc mockMvc;

    @Test
    void sendsNotificationAndReadsItBack() throws Exception {
        MvcResult created = mockMvc.perform(post("/api/v1/notifications")
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                {
                    "userId": "user-42",
                    "title": "Money debited",
                    "content": "INR 5,000.00 debited from A/c XX1234.",
                    "notificationType": "SMS",
                    "priority": "HIGH",
                    "idempotencyKey": "txn-9001:debit"
                }
            """))
    }
}

```

```

        }
        """))
        .andExpect(status().isCreated())
        .andExpect(header().string("Location", startsWith("http://localhost/api/v1/notifications/")))
        .andExpect(jsonPath("$.status").value("SENT"))
        .andExpect(jsonPath("$.content").value(startsWith("INR 5,000.00 debited from A/c XX1234.")))
        .andReturn();
String id = JsonPath.read(created.getResponse().getContentAsString(), "$.notificationId");

mockMvc.perform(get("/api/v1/notifications/{id}", id))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.notificationId").value(id));

mockMvc.perform(get("/api/v1/notifications").param("userId", "user-42"))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$[*].notificationId", hasItem(id)));

// Same idempotency key again: same notification, nothing new is sent.
mockMvc.perform(post("/api/v1/notifications")
        .contentType(MediaType.APPLICATION_JSON)
        .content("""
            {
                "userId": "user-42",
                "title": "Money debited",
                "content": "retry of same event",
                "notificationType": "SMS",
                "priority": "HIGH",
                "idempotencyKey": "txn-9001:debit"
            }
        """))
        .andExpect(jsonPath("$.notificationId").value(id));
}

@Test
void invalidRequestReturns400WithErrors() throws Exception {
    mockMvc.perform(post("/api/v1/notifications")
        .contentType(MediaType.APPLICATION_JSON)
        .content("""
            {
                "userId": "",
                "title": "Hi",
                "content": "Body",
                "priority": "LOW"
            }
        """))
        .andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.errors").isArray());
}

@Test
void unknownNotificationReturns404() throws Exception {
    mockMvc.perform(get("/api/v1/notifications/{id}", "does-not-exist"))
        .andExpect(status().isNotFound());
}
}

```

15. Common mistakes

Before you compare your code line by line, check it against these. Each one is handled somewhere in the reference code above.

MISTAKE	WHAT GOES WRONG	DO THIS INSTEAD
Engine uses <code>switch (type)</code> to pick a channel	Every new channel edits the engine	Inject <code>NotificationChannelResolver</code>
<code>publish()</code> has no try/catch per listener	One broken listener stops delivery	Catch and log around each <code>onEvent</code>
Saving <code>SENT</code> inside the same try/catch as <code>send()</code>	A DB error re-sends the SMS	Update the status after the try block
<code>Instant.now()</code> called directly in code	Tests depend on the real time and become flaky	Inject a <code>Clock</code> bean
Passing the live <code>Notification</code> to listeners	A listener can change its status	Pass an immutable <code>NotificationEvent</code>
Logging <code>notification.getContent()</code>	OTPs and balances end up in log files	Log only ids and types
<code>@Enumerated</code> without <code>EnumType.STRING</code>	Reordering the enum corrupts stored data	Always use <code>EnumType.STRING</code>
Checking the idempotency key only in Java	Concurrent duplicates slip through	Add a unique DB constraint too
<code>@Data</code> on the JPA entity	Broken <code>equals</code> / <code>hashCode</code> , unwanted setters	<code>@Getter</code> + <code>@Builder</code>
<code>@Transactional</code> on the whole <code>send()</code>	DB connection held while sending and sleeping	Short writes, no outer transaction
<code>@Autowired</code> on fields	Hidden dependencies, hard to unit test	Constructor injection
Retrying every exception	Bugs retried three times, delays wasted	Retry only retryable delivery failures